

Maths & Comp

24COC255

F221611

**OTTOMAN EXCHANGE:
A LOAN PLATFORM WHICH
USES A SMART CONTRACT
WITH RATE SWITCHING FUNCTIONALITY**

by

Abubakar Othman

Supervisor: Alia Asheralieva

*Department of Computer Science
Loughborough University*

June 2025

Abstract

This project aims to introduce a decentralised Ethereum blockchain application that allows users to choose between a fixed or floating interest rate. The switch system will allow them more flexibility, making it favourable. The system comprises an innovative contract architecture written in Solidity integrated with Chainlink Oracle to provide live ETH/USD price feeds.

The front was developed in React.js and styled with TailwindCSS using the Soft UI Dashboard theme, which uses Wagmi v2 and Rainbowkit to allow for seamless wallet connection and easy contract read/writes. The platform supports the full loan lifecycle from funding to borrowing to repayment or liquidation.

With a scalable and transparent project, the project aims to provide users with an easy-to-use flexible loan management tool that contributes to the fast-growing DeFi ecosystem.

Key Words: Decentralised Finance, Blockchains, Cryptocurrency, Smart Contracts, Oracle, Utilization, Ethereum, Sepolia, MetaMask, Chainlink, Collateral.

Table of Contents

1. Introduction	5
1.1. Motivation.....	5
1.2. Contributions	5
2. Literature Review	6
2.1 Decentralised Finance (DeFi)	6
2.2. Stablecoins in DeFi	7
2.3. Lending and Borrowing in DeFi.....	8
2.4. Interest Rate	9
2.5. Smart Contract Technology	11
2.6. Blockchain Technology	12
3. Software Selection.....	14
3.1 Integrated Development Environment (IDE).....	14
3.2. Blockchain Deployment Framework.....	14
3.3. Smart Contract Language	15
3.4 Frontend Framework	15
3.5. Blockchain Network.....	15
3.6. Oracle Integration	16
3.7 Testing Frameworks	16
3.8. Deployment Tools.....	17
3.9. Package and Dependency Management.....	17
3.10 Version Control and Collaboration	17
3.11 Wallet Integration.....	17
4. Design and Architecture	19
4.1. High-level System Architecture	19
4.2. Smart Contract Architecture	21
4.3. Frontend and Wagmi Integration.....	24
4.4. Oracle Integration and Interest Calculation Logic.....	27
5. Implementation	29
5.1. Core Functionality.....	29
5.2. Smart Contract Development	29
5.3. Frontend Implementation	34
5.4. Oracle Implementation.....	39
5.5. AI Assistant Implementation	40
5.6. Problems Encountered and Solutions	43

6. Testing and Deployment.....	44
6.1. Unit Testing.....	44
6.2. Deployment to Sepolia Testnet	44
6.3 Integration and End-to-End Testing.....	45
7. Results and Evaluation	52
7.1. Functional Evaluation	52
7.2. Performance and Optimisation	55
7.3. User Experience and Interface	58
7.4. Security and Reliability.....	59
8. Limitations and Challenges.....	59
8.1. Technical and Functional Limitations	59
8.2. Project Challenges and Obstacles	60
9. Conclusion	61
9.1. Summary of Achievements.....	61
9.2. Learning and Insights	61
9.3. Potential Additions	62
9.4 Closing Thoughts	62
Appendix	63
Appendix A: Code snippets	63
Appendix B: UI Screenshots	71
Appendix C: Detailed Test Results	71
Appendix D: Deployment Scripts and logs	72
Appendix E. Workplan Summary	72
Appendix F. Themes and References	73
Appendix G. Formulas	73
Bibliography	74

1. Introduction

In this project, I wish to create a decentralised finance smart contract that uses rate-switching loans. The smart contract will allow for a dynamic interest rate, allowing borrowers to toggle between fixed and floating interest rates based on market conditions. It will give users better control over their loan costs whilst maintaining transparency and security for users. The rate switching will be beneficial in choosing the best interest rate structure depending on the prevailing market conditions.

The smart contract will ensure that floating rates align with market conditions by integrating decentralised oracles for real-time data from major DeFi lending platforms or external markets. The Oracle will always provide accurate market conditions and both lenders' and borrowers' accurate and fair loan terms.

1.1. Motivation

As DeFi is trending and will continue to grow exponentially, it will inevitably become part of our daily lives. It provides a different solution to banking without the need for a bank. Decentralised Finance also comes with a range of benefits, such as:

- **Permissionless** – allowing for anyone with a crypto wallet and internet access to make use of DeFi applications built on Ethereum
- **Immutable** – This means there is tamper-proof data coordination across the Blockchain's decentralised structure, increasing security.
- **Programmable** – Since smart contracts are programmable, banking can be automated.

These, coupled with a range of benefits, make the technology highly desirable and will play a vital part in the future.

With my project, I aim to provide something that can benefit users who wish to borrow money by allowing them to pick their best rates.

1.2. Contributions

I used a combination of blockchain technologies and programming languages to build my DApp. The technology selection included Solidity, the primary language for writing smart contracts on the Ethereum blockchain. A decentralised Oracle such as Chainlink provided reliable and secure data for Solidity on the backend services to support the front end. The frontend framework, React, was used to communicate with the Ethereum blockchain and allow interaction with the smart contract.

2. Literature Review

2.1 Decentralised Finance (DeFi)

2.1.1. What is DeFi

Decentralised Finance is the fast-growing peer-to-peer solution to traditional Finance. DeFi leverages Blockchain technology and cryptocurrencies to create a decentralised and transparent financial system. DeFi operates autonomously through smart contracts; this removes the need for third parties, allowing for cheaper and faster transaction times.

2.1.2. Will DeFi take over Finance

DeFi offers multiple benefits over traditional Finance, which makes it desirable. Unlike traditional Finance, which relies on third parties such as banks, DeFi offers autonomously through smart contracts facilitating peer-to-peer interactions. This approach promotes financial inclusion, allowing more individuals globally to access digital banking services without dealing with geographic or socioeconomic barriers. Platforms also operate 24/7, allowing anyone to use the services at any time, unlike traditional Finance.

DeFi is also much more transparent because it is built on public blockchains, meaning all transactions are public and visible to anyone who wishes to see them. This transparency will give users insight into how funds are managed, how liquidity pools are created and how the contract is executed. This is different from traditional banking, where a third party such as the bank will keep you in the dark regarding loans, fees and fund management.

Instead of disrupting traditional banking, blockchain technology has caused interest in traditional banks looking to modernise services, such as JPMorgan Quorum, which was created as an enterprise blockchain solution to provide faster, more efficient services whilst retaining regulatory compliance.

So, instead of "replacing" traditional banking, it has been seen as a complement with financial institutions partnering with DeFi platforms to offer decentralised services.

2.1.3. Characteristics of DeFi

General Characteristics of DeFi are that it is:

- **Decentralised** – Operates autonomously without centralised control
- **Permissionless** – Anyone can create a DeFi app if they have internet access
- **Transparency** – Code and transactions on the Blockchain are visible to anyone
- **User-Centric** – Users are rewarded for participating in the service, and users can choose how the service is accessed
- **Interoperable** – DeFi services are interoperable due to the shared blockchain network where they reside, and cross-blockchain interoperability reinforces this.

Table I: DeFi vs CeFi

Decentralised Finance	Centralised Finance
Funds held by the user	Funds held by third party
No customer Service	Customer service provided by third-party
Anonymous use available	Anonymous use unavailable
Permissionless	Requires Permission
Low transaction fees	High Transaction fees
Sometimes, complex UI/UX	Simple UI/UX
Fully transparent	Hides info from the user
Less security	More security
Auger, Kyber, MarkerDao	Coinbase, Libra, BlockFi

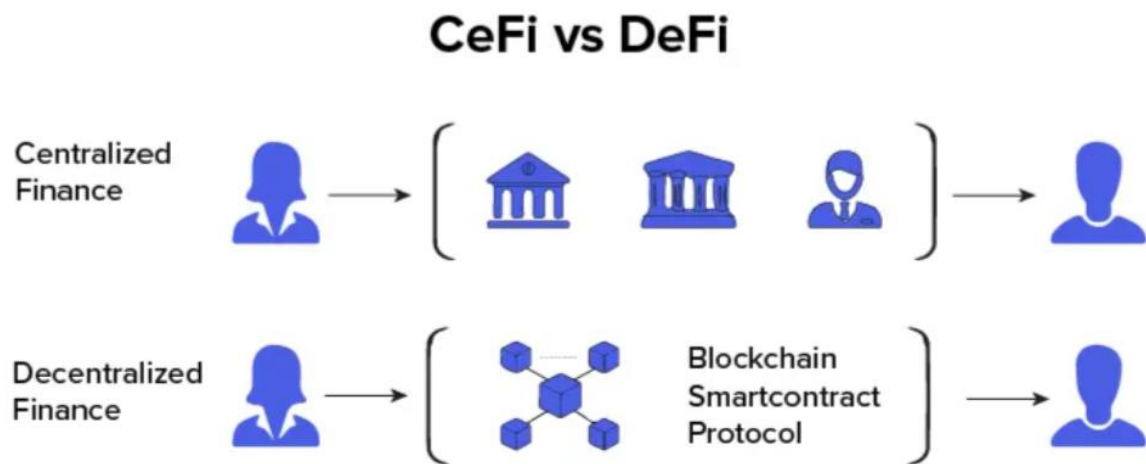


Figure 1: CeFi vs DeFi (Source: <https://appinventiv.com/blog/defi-vs-cefi/>)

2.2. Stablecoins in DeFi

2.2.1. What are Stablecoins

Stablecoins are cryptocurrencies whose values are pegged or tied to the value of another currency, commodity, or financial instrument. They provide an alternative to the high volatility of popular cryptocurrencies such as Bitcoin (BTC), which have made crypto investments less suitable for everyday interactions. They bridge the gap between fiat currencies like the US dollar and cryptocurrencies. Due to them being price-stable digital assets like fiat currencies but having the utility of cryptocurrencies, they are less volatile than popular cryptocurrencies.

Popular stablecoins are USDT, USDC, and DAI, each of which has its own unique properties.

2.2.2. Role of Stablecoins in DeFi

Stablecoins play a huge role in DeFi as they are central to its functioning and growth. They help make the volatile world of cryptocurrencies more stable by making them less volatile. Being less volatile makes it a great choice for people who want to buy or trade using DeFi applications because they do not have to worry about price changes.

Stablecoins are easy to trade for other crypto and traditional currencies. They work as a bridge between the two different types of currencies due to their stability. They help provide liquidity as they are a stable asset.

There are four distinct types of stablecoins used in DeFi:

- **Fiat-Backed** – The most popular type where the collateral isn't another cryptocurrency. This means fiat currency is in reserve as collateral. The collateral remains with a central issuer or financial institution.
- **Crypto-Backed** – Backed by another cryptocurrency as collateral, and the process occurs via a smart contract rather than having a central issuer.
- **Commodity-Backed** – Collateralized using physical assets like metal, oil and real estate
- **Algorithmic Stablecoins** – Don't use fiat or crypto as collateral. Instead, their price stability results from using specialised algorithms and smart contracts that manage the supply of tokens in circulation

2.3. Lending and Borrowing in DeFi

2.3.1. What is Lending and Borrowing in DeFi

Lending and Borrowing in DeFi allow any user to borrow and lend digital assets through decentralised protocols governed by smart contracts, which will determine terms such as interest rates, transaction amounts, repayment terms and expiration dates.

2.3.2. How Does Lending Work

Lending works by:

1. Choose a DeFi Platform such as [28] Aave or [29] MakerDAO
2. Connect your Crypto wallet to the platform
3. Transfer Crypto assets to the liquidity pool
4. Approve transaction through your wallet to complete the deposit
5. Assets earn interest based on the supply and demand dynamics of the pool

2.3.3. How Does Borrowing Work

Borrowing works by:

1. Choose a DeFi Platform such as [28] Aave or [29] MakerDAO
2. Connect Crypto wallet to the platform
3. Decide between a collateralised loan and a flash loan
4. For collateralised loans, deposit collateral into the platform
5. Enter the amount you wish to borrow
6. Confirm amount
7. Platform transfers borrowed amount

2.3.4. DeFi Lending and Borrowing Pros and Cons

Lending and Borrowing on DeFi comes with a range of advantages and disadvantages

Table II: Advantages and Disadvantages of Blockchains for DeFi Applications

Advantages	Disadvantages
You get full control over finances.	Changes made to the Blockchain are irreversible.
The number of people involved is minimal	Hard to understand
Low cost	Smart Contract Vulnerability
Resilient to Failures due to being built on Blockchain	
24/7 Access	
Resistant to Hacking attacks	
No censorship	
Ensure anonymity	

2.4. Interest Rate

On DeFi, interest rates are set automatically using an algorithm coded into a platform's smart contract. The rates are based on the amount deposited and borrowed for the crypto asset. The rate is also known to all users. The formula for this is the borrowing rate, which is an increasing function of the utilisation rate, which is the ratio between the amount borrowed and the amount deposited. Interest rates fluctuate depending on the supply and demand for tokens and are constantly adjusted based on market demand, liquidity of assets and more factors. Rates can really vary and be as high as 80% if there is an imbalance between borrowers and lenders, as seen in the graph below.

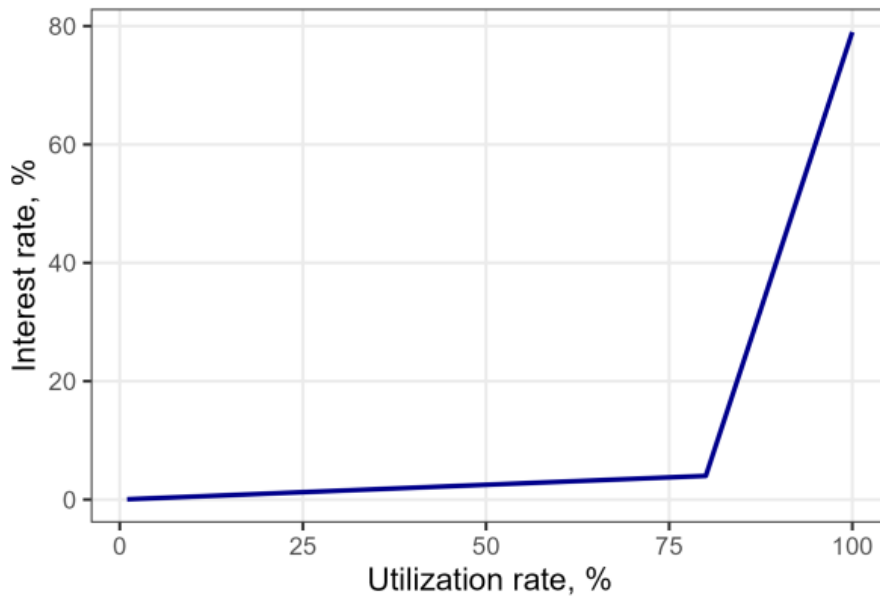


Figure 3: Title. Source: Aave v2 smart contract for USDC stablecoin

2.4.1. Annual Percentage Yield and Annual Percentage Rate

Two key metrics for measuring interest in crypto are APY (Annual Percentage Yield) and APR (Annual Percentage Rate). Both measure interest in various types of crypto activities, but they represent different concepts. They are calculated differently, leading to varying results.

- **APY (Annual Percentage Yield)** – Represents the rate of compensation earned on financial activity in the year, factoring in the effects of compounding compensation. This is commonly used in DeFi platforms. Suitable for those comfortable with managing the complexities of compounding, compounding can boost returns. This allows capital to increase over time. To calculate APY :

$$APY = \left(1 + \left(\frac{r}{n}\right)\right)^n - 1 \quad (1)$$

- **r** - nominal interest rate
- **n** - number of compounding periods in a year
- **APR (Annual Percentage Rate)** – A commonly used financial metric that represents the annual rate for financial activities or loans. Commonly used in traditional Finance but is starting to be more used in DeFi. Suitable for those who want a clear view of potential interest earnings. Provides a method for comparing different financial opportunities without considering the compounding effect on interest. Calculated as a simple rate, meaning it doesn't account for previously gained compensation. To calculate APR:

$$APR = \left(\frac{r}{n}\right) * n \quad (2)$$

- **r** - nominal interest rate expressed as a decimal
- **n** - number of compounding periods in a year

Both APY and APR have a place in assessing and comparing financial opportunities. Understanding them will help make informed financial decisions.

2.4.2 Utilisation in DeFi

Utilisation is key in DeFi platforms such as Aave to determine interest rates. It is calculated as:

$$Utilization Rate = \frac{Total Borrowed}{Total Liquidity} \quad (3)$$

As utilisation increases, there is a higher demand for loans, so interest rates adjust accordingly to maintain liquidity. With this, we can determine our interest rate as follows:

$$Interest Rate = Base Rate + Multiplier * Utilization Rate$$

2.5. Smart Contract Technology

2.5.1. What Are Smart Contracts

Like any contract, a smart contract consists of the terms of an agreement. What differs is that a smart contract's terms are executed as code running on a blockchain such as Ethereum. This allows developers to build DeFi apps, taking advantage of the many benefits of Blockchain. They are behind the idea of having transactions without the "middleman" by making it possible to automate transactions and decentralise any sort of deal, no matter how complex it may be.

Smart Contracts were designed to reduce the costs needed in services such as banking by automating any kind of exchange. A smart contract can be developed and deployed by anyone whilst being visible to everyone, allowing parties to see what logic the contract follows. Smart contracts are written in a range of languages such as Solidity, Web Assembly and Michelson.

2.5.2. Importance of Smart Contracts

Smart Contracts allow developers to build a variety of DeFi applications. They are now used in a range of technologies, such as financial tools and game experiences. Smart contracts have the potential to radically change a range of industries, such as supply chain, banking and more. They solve the problem of trust by making it more honest, secure and fair. Smart contracts are the most effective solutions in simplifying businesses through the range of benefits provided

Table III: Benefits and Limitations of Smart Contracts

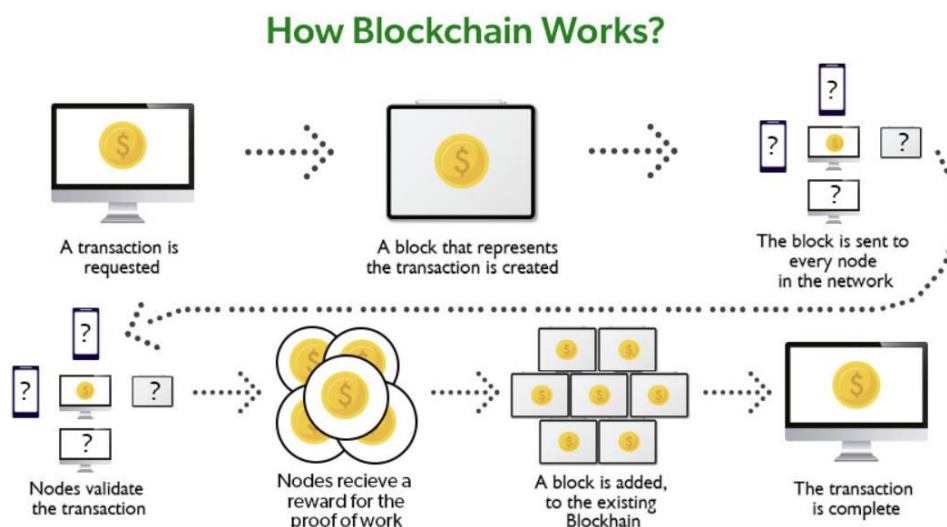
Benefits	Limitations
Autonomy – The third party needs to be reduced	Third-Party – Still involved but will fill a different role
Backup – All documents stored on Blockchain are recoverable no matter what	Hard to change – Since it is all coded, it is expensive and time-consuming to change if needed
Speed – Since it is automated, hours are saved in contrast to contacting businesses	Vague – Smart contracts can sometimes be hard for parties to understand due to vague terms.
Accurate – Eliminate human error	

2.6. Blockchain Technology

2.6.1. What is Blockchain

Blockchain is a distributed database shared across a computer network. Data is stored in blocks which are linked together via cryptography. They play a crucial role in Decentralised Finance by providing a secure and decentralised record of transactions. What makes it secure is that it is not able to change data once it is on the Blockchain. But when an error occurs, it cannot be reversed. Instead, you add a new transaction to reverse the error, and then both are visible. Since it cannot be changed, the only trust needed is where the user or program enters data. Different types of information can be stored on a blockchain, but the most common use is as a transaction ledger. Smart contracts are stored on the Blockchain and run automatically.

2.6.2. How does Blockchain Work



Source: GeeksForGeeks <https://www.geeksforgeeks.org/how-does-the-blockchain-work/>

2.6.3. Types of Blockchain

There are four types of blockchains:

- **Public Blockchain** – There are no restrictions, and it is open to the public; it is not owned by anyone
- **Private Blockchain** – Only selected nodes can participate, making it more secure but less public.
- **Hybrid Blockchain** – Mix of public and private where some parts are controlled by an organisation and the rest is public.
- **Consortium Blockchain** – More than one organisation manages the Blockchain, where the Blockchain validates the transaction and initiates or receives transactions.

Table IV. Benefits and Limitations of Blockchain

Benefits	Limitations
Decentralised	Hard to scale
Secure	Lack of Regulations
Efficient	Gas Consumption
Innovative	

3. Software Selection

This section will discuss the tools and frameworks used for the development of the Loan-Switching Smart Contract.

This section is still subject to change and will likely change in the future, but it reflects what is used up to the point of reading this.

Table V. Technologies Chosen and Use

Use	Technology Chosen
IDE	Visual Studio Code
Testing and Deployment	Sepolia, Javascript, Hardhat
Testing Framework	Sepolia
Blockchain	Sepolia, Ethereum
Backend	Solidity
Front end	Javascript (React, Wagmi, Ethers)
Libraries	React
Oracle	ChainLink
Wallet Integration	MetaMask, Rainbowkit, Wagmi

In this table, I highlighted the technologies I had chosen for each part of the system.

3.1 Integrated Development Environment (IDE)

For the IDE, I decided to use Visual Studio Code due to my familiarity with it. It is also arguably the most popular open-source code editor. The IDE also boasts a range of features which make it more convincing to use, such as:

- The vast library of extensions
- The built-in terminal allows me to use the terminal without leaving the IDE
- Debugging support which will help me troubleshoot my code more easily

When looking into IDEs to use, I was also considering Remix IDE, which is popular for developing smart contracts, especially for Ethereum. In the end, I decided to go with Visual Studio Code because it is not web-based and has offline functionality.

3.2. Blockchain Deployment Framework

For testing, I will rely on Hardhat to manage test cases and deploy the smart contract on Sepolia for testing and Ethereum for the final product. This is because it comes with an automated testing framework, which will make testing the contracts easier. I will use JavaScript due to its compatibility with Hardhat and my past experience in the language. I will be able to cover different testing scenarios

using JavaScript. Using Hardhat will simplify the deployment of my contract to live Ethereum networks.

I decided to use Hardhat over Truffle, which is another popular blockchain tool because I personally found Truffle easier to use than Hardhat and came across fewer errors. Also, due to Truffle being sunset, Hardhat is better for the long term.

3.3. Smart Contract Language

Initially, I had planned to use Python due to it being arguably my strongest programming language. As I researched more about Blockchains, I got to see how much better it would be to use Solidity due to its being an industry-standard language for writing smart contracts on the Ethereum blockchain. This is due to its being specially designed for EVM chains and having built-in Blockchain support features.

I used Solidity 0.8.13 over the more recent version due to it being less restrictive.

3.4 Frontend Framework

I used React.js, which is ideal for web apps. It has allowed for a platform which is intuitive for lenders and borrowers to interact with the smart contract. It also leverages hooks, which will be very useful for state management and blockchain interaction.

I have also used Wagmi, which is a react hooks library that greatly simplifies interactions with the Ethereum blockchain, especially for Dapps. It allowed for an intuitive way of handling loan interactions on the front end, allowing me to focus on the logic of the Dapp rather than the complexity of blockchain interactions, significantly streamlining the process.

I also made use of the Soft UI Dashboard React Template to speed up the frontend development process and allow for a nicer user experience.

<https://www.creative-tim.com/product/soft-ui-dashboard-react>

3.5. Blockchain Network

I used the Ethereum Sepolia Testnet, which allowed me to test my application and smart contracts in a realistic and safe environment before deploying onto the Ethereum mainnet. It provided a mainnet testing experience that allowed me to experiment with my code without risking real funds. It also has a predictable network state.

For actual Dapp, I used the Ethereum mainnet, which is used for real-world transactions and deployment of actual Dapps that use the Ethereum Blockchain. It is also the original and primary network where real Ethereum transactions occur. It also facilitates the execution of smart contracts.

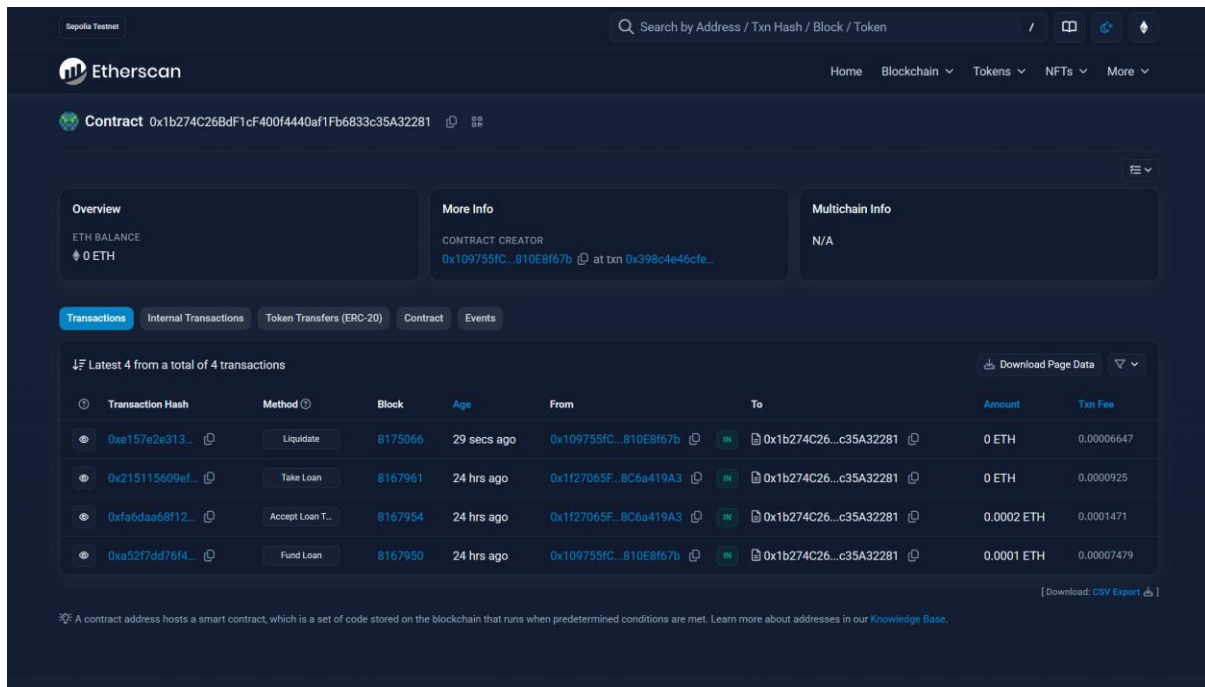


Figure 3.1 Etherscan view: Here, I am showing what a smart contract looks like on the Etherscan website

3.6. Oracle Integration

Chainlink is being used due to its decentralised tamper-proof price feeds. I am using Oracle to fetch real-time off-chain data for ETH and price feeds to ensure the most accurate values are used. It provides data for the floating rate, allowing the borrower to get the most accurate rate, which will help determine whether to switch rates.

3.7 Testing Frameworks

I have used a range of testing frameworks in different stages of development.

Early on, before I implemented my front end, I used Hardhat to do my testing on the command line, and Etherscan was used to confirm that the transactions have been done currently. This made it so I wouldn't have to constantly look at both accounts and just be able to look at smart contracts, making it easier to go through.

During frontend deployment, I used the front end for the testing and was able to do the testing using actual wallets and Wagmi.

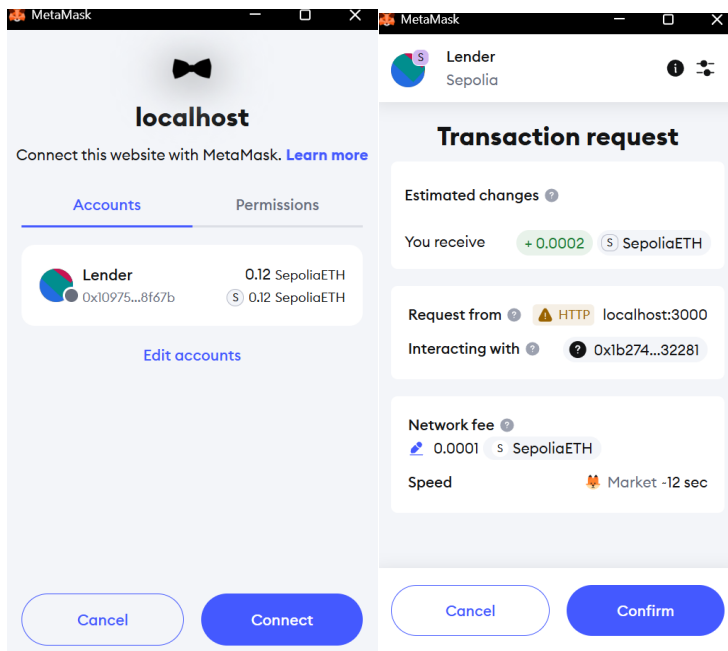


Figure 3.2: Here, I am showing screenshots of how a user can connect to MetaMask and how they can confirm transactions.

3.8. Deployment Tools

I had initially used Truffle but phased it out.

I will be using Hardhat to deploy the LoanFactory, which deploys the LoanRequest contracts.

```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat run scripts/deploy-factory.js --network sepolia
Deploying contracts with the accounts: 0x109755fC6Bf1A294D1bCC95C08911a7810E8f67b
LoanFactory deployed to: 0xA3BcaC7FACB62700B1509AE8Cd9Bfb5964A1dA2
```

Figure 3.3: A deployment of my LoanFactory using Hardhat on the command line

3.9. Package and Dependency Management

The node package manager (NPM) was used to manage dependencies on the front and backends, including Wagmi v2, React, Hardhat, Ethers.js, Tailwind, and more.

3.10 Version Control and Collaboration

Git has been used for source control to ensure traceability and backup, as well as GitHub

3.11 Wallet Integration

I made it so MetaMask is the primary wallet used to interact with deployed smart contracts on Sepolia and Ethereum. Users can connect their wallets through Rainbowkit and Wagmi. This allowed for secure wallet-based authentication and transaction signing.

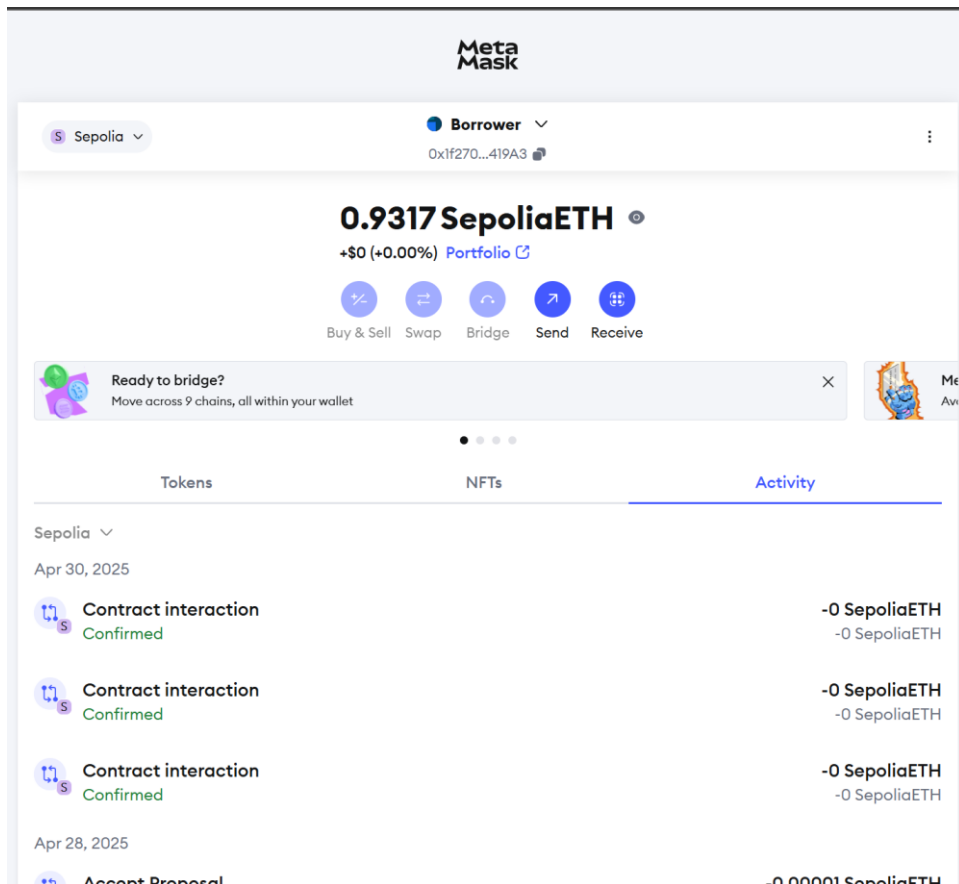


Figure 3.4. Full view of MetaMask wallet where a user can see their transactions

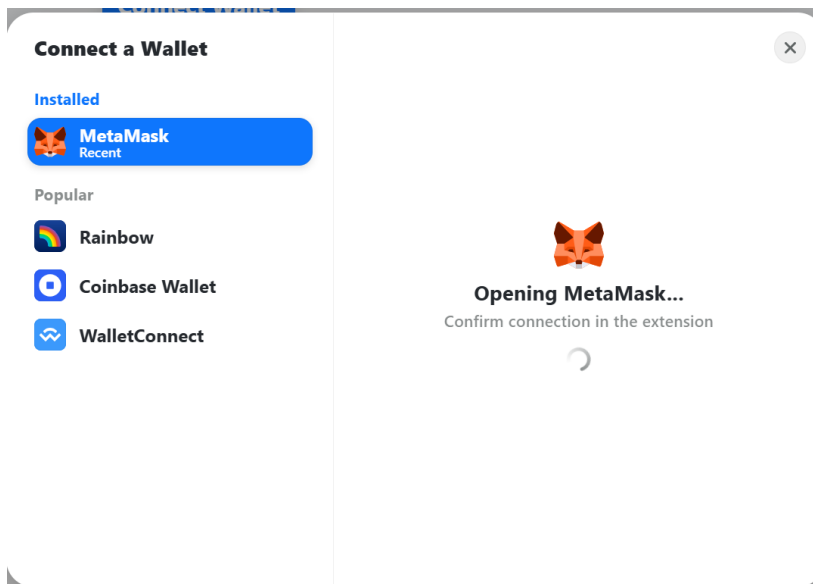


Figure 3.5 Rainbowkit view where a user can connect their wallet, in particular, MetaMask

MetaMask allowed users to fund loans, accept terms, repay, switch rates and confirm transactions. It is also very easy to use as there is an app and Chrome extension.

4. Design and Architecture

This section goes into the design and architecture of the rate-switching smart contract, highlighting how the various components are implemented. It explains the reasoning behind the decisions I made and how it all comes together to produce the application. It also talks about the front end and its various pages.

4.1. High-level System Architecture

Table VI. The technology chosen for Layer

Layer	Technology	Use
Frontend	React.js, Tailwind.css,MetaMask, Wagmi v2	Allows for an intuitive user interface for users to interact with the DeFi platform and allows for secure user authentication and wallet integration
Blockchain Interaction	Wagmi v2 Hooks	Allows for blockchain communication, allowing the frontend to read data from the backend
Backend Layer	Solidity	Allows for the management and creation of loans handling core functions
Blockchain Layer	Chainlink Oracle, Sepolia/Ethereum Blockchain	Allows for secure price feeds and execution of transactions and safe storage

In this table, I have summarised what technologies I have used and what role they have played in the system.

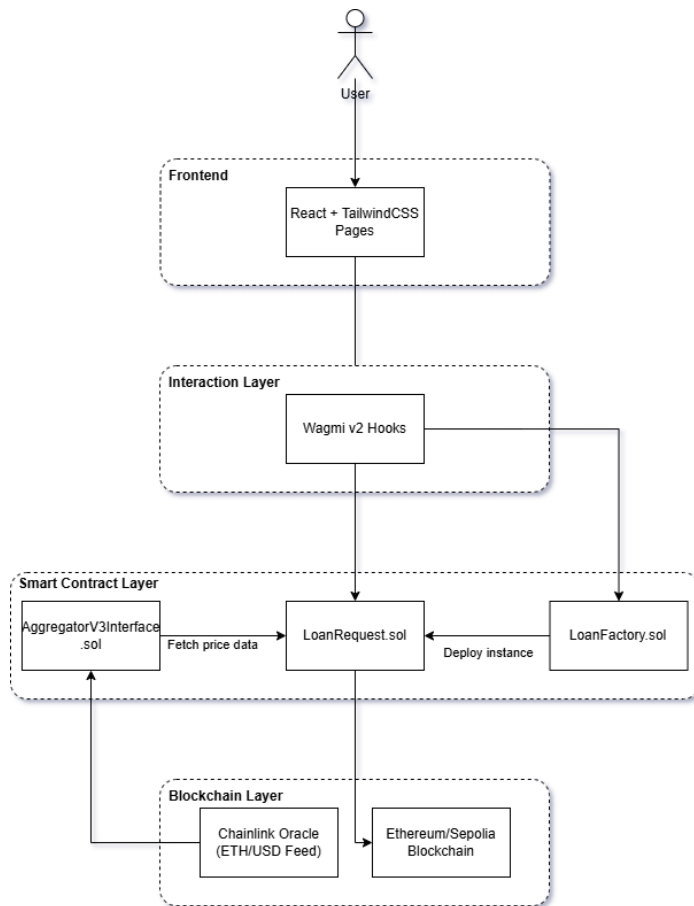


Figure 4.1: System Architecture diagram of my Dapp and what connects to what.

4.2. Smart Contract Architecture

4.2.1. Class Diagram



Figure 4.2: A class diagram highlighting how my different smart contracts interact with each other

4.2.2. Function uses

LoanFactory.sol

- Manages all the loans made on the program.
- Stores the proposals and allows for smart contracts to be created where transactions will take place.
- Use of a lot of required conditions for security.
- This is the contract where the utilisation calculations are done.
- Deploys LoanRequest contracts
- Map borrower to lender

Table VII: LoanFactory functions

Function	Use
createProposal	Creates borrower's loan proposal based on the conditions they had specified
getAllOpenProposals	Returns all proposals that haven't been accepted by a lender
acceptProposal	Allows the lender to accept the proposal and create a LoanRequest contract, which gets deployed on the Blockchain. The Loan Request gets stored.
getProposal	Returns a proposal based on the proposal id
getAddressForProposal	Returns the load address for the proposal
getAllLoans	Returns all the factory's loans
isFactoryLoan	Check if a loan belongs to our loan factory.
recordBorrower	Registers the borrower onto our records
getsLoansByLender	Gets all the loans where the user is lending
getsLoansByBorrower	Gets all the loans where the user is borrowing

In this table, I give a brief overview of functions I have used for LoanFactory.sol.

LoanRequest.sol

- Manages the main lending and borrowing process between a lender and borrower.
- Uses a lot of required conditions for security.
- Here is the contract where the APR and APY interest rates are calculated.
- State transitions are handled here.

Table VIII: LoanRequest functions

Function	Use
fundLoan	Allows the lender to deposit the ETH amount agreed on for lending. Transitions Loan to Funded state
acceptLoanTerms	Allows borrower to accept the loan terms. Transitions loan to accepted state
takeLoan	Allows borrower to take the loan. Transitions loan to the taken state
repay	Allows borrower to repay the loan. Transitions loan to repaid state.
liquidate	Allows the lender to liquidate the loan. Transitions loan to liquidated state
updateRates	Will update the floating rate and fixed rate so that it is up to date. It also calculates the fixed rate based on the floating rate.
getFixedRates	Returns fixed rate when called
getFloatingRates	Returns floating rate when called
switchRateType	Switches rate type to the other one when called
calculateInterest	Will calculate the interest that has to be paid at the end of the loan
getLoanState	Will return the loan's current state
getLoanAmount	Will return how much was agreed for the loan
getFeeAmount	Will return the fee for the loan
getEthCollateralAmount	Will return the collateral amount
getRepayByTimestamp	Will return the repayment date

In this table, I give a brief overview of functions I have used for LoanRequest.sol.

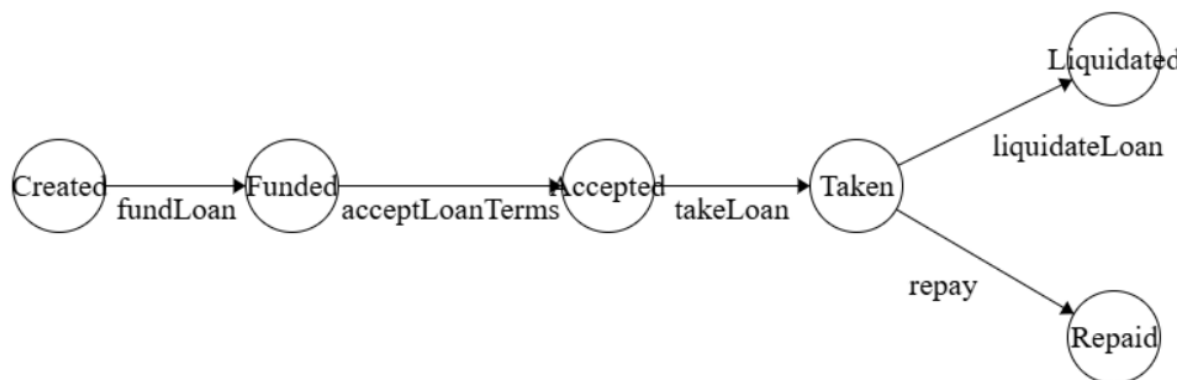


Figure 4.3: State Transition Diagram highlighting how the states of my LoanRequest smart contract will change as the system goes on.

[AggregatorV3Interface.sol](#)

I plan to use the AggregatorV3Interface from the DevHub website. <https://docs.chain.link/data-feeds/api-reference>

It is used to help fetch the Chainlink oracle and turn it into data we can use

Table IX: AggregatorV3Interface functions

Function	Use
decimals	Gets the number of decimals present in the response value
description	Get a description of the underlying aggregator that the proxy points to
version	Gets the version representing the type of aggregator the proxy points to
getRoundData	Get data about a specific round using the roundID
latestRoundData	Get data from the latest round.

In this table, I give a brief overview of functions I have used for AggregatorV3Interface.sol.

4.3. Frontend and Wagmi Integration

Table X: Component structure with the proposed framework

Component	Use
Dashboard	Provides current ETH data, allows users to log into the platform and also has an AI assistant where the user can ask the system what they would like
Create Proposal	Allows the user to put on their proposal specifying details such as how much they want to be lent to them as well as how much collateral they want to put down. Here is also where they pick the calculation type.
Proposal Explorer	Allows users to go through all proposals and accept loans which they choose. Fitted with an AI assistant which can break down the proposal and help choose
My Loans	Allows users to go through Loans they funded and liquidate if the time has passed, as well as ask the assistant their opinion on the loan
My Borrowed Loans	Allows users to go through Loans they borrowed, repay and switch rates, as well as ask the assistant their opinion on the loan
Navbar	Allows users to switch between pages, although some cannot be accessed until they sign in.

Here, I explain what each page is for and the part played in the system.

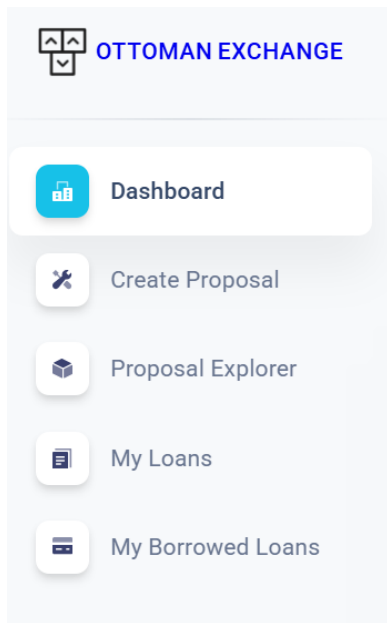


Figure 4.4: Navbar and all the pages within it

Table XI: Used Wagmi Hooks

Hook	Use
useWriteContract	Allows to write onto the solidity contract
useReadContract	Allows us to read from the solidity contract and returns what we want to read
useWaitForTransactionReceipt	Waits for transactions to be included on the block
user account	Gets the current signed-in account

Used Wagmi Hooks and what they were used for

4.3.3 UX Features

I will make it so you can see the transaction status in MetaMask and on the UI

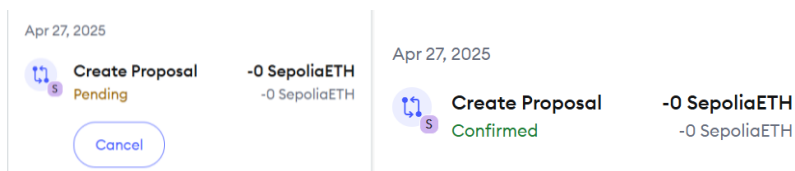


Figure 4.5: Showing how the transaction status changes depending on what is happening

Can easily connect to your wallet from the start as well. The AI assistant is also easily accessed as it is on the side, allowing for an intuitive user interface.

Loan Assistant

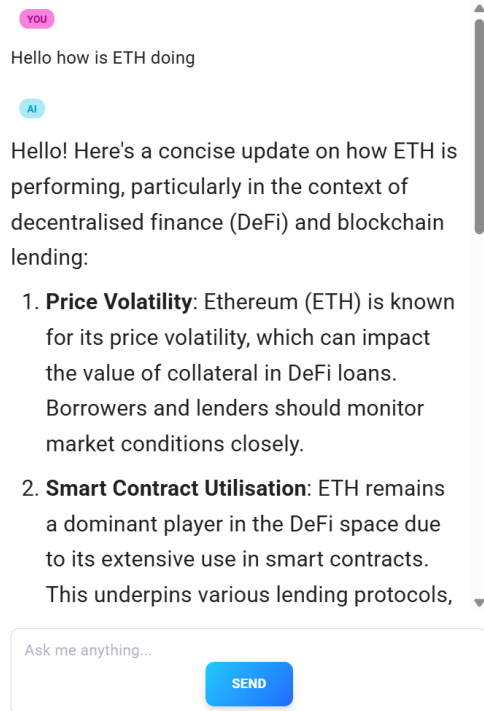


Figure 4.6: Loan Assistant on the Dashboard

When asking for a connection, a popup will appear, prompting the user to connect to where they can use their Metamask wallet. There is also the option to use other wallets

such as Rainbow, Coinbase Wallet and WalletConnect

but I have decided to focus on MetaMask for the demonstration

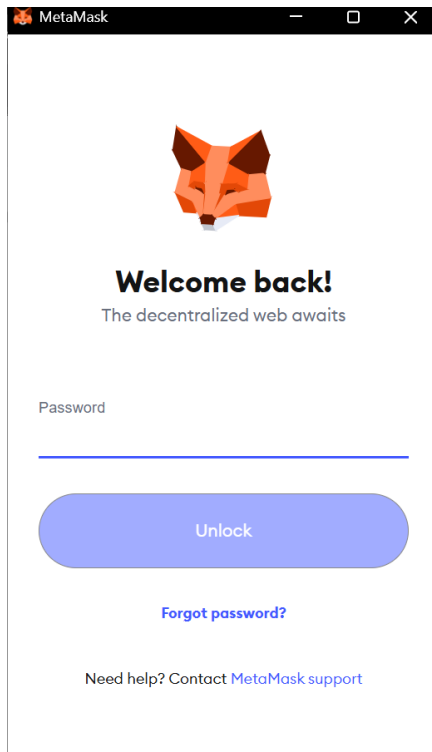


Figure 4.7: MetaMask sign-in from extension

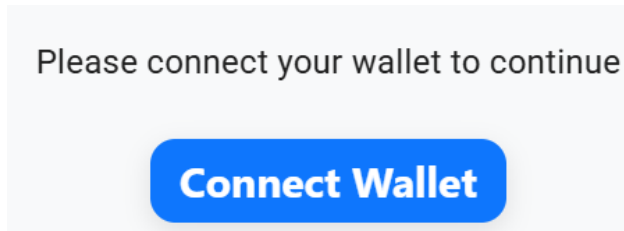


Figure 4.8: Connect Wallet button from RainbowKit

The front end was styled using TailwindCSS for rapid response. Also used Soft UI Dashboard React to get a modern aesthetic

4.4. Oracle Integration and Interest Calculation Logic

4.4.1. Chainlink Oracle Integration and Reliability

The contract uses the `AggregatorV3Interface` to pull ETH/USD data from Chainlink. This ensures my floating rate is tied to real-world ETH prices, increasing trust in the system. Here is the address I will use:

0x8e604308BD61d975bc6aE7903747785Db7dE97e2

Chainlink uses multiple nodes and aggregators, allowing for a smaller risk of failure. There is also a small gas fee for reading from Chainlink, making it a good deal for the security given.

4.4.2. Interest Rate Switch Logic

At the start, a fixed rate will be calculated based on the price feed at the time. The rate will be a good bit higher than the floating rate but will be fixed, meaning it won't change the whole loan term, allowing for a more predictable payment.

The floating rate is given by the Chainlink oracle, so the user will pay depending on the market at the end. It will be less predictable but could work out better if lucky.

4.4.3 Loan Repayment Calculation

The repayment amount will be dependent on a range of factors:

- The interest calculation type chosen at the start will be APR or APY
 - If APR is used, use formula (2)
 - If APY is used, use formula (1)
- The amount of Utilisation was calculated from the amount the system used. As the utilisation rate increases, so does the floating and fixed rates. Use formula (3)
- The rate type that was picked, if floating then, will be dependent on the market at the time of payment.

5. Implementation

This section will highlight the current progress which has been made, as well as problems and what will be addressed moving forward.

5.1. Core Functionality

The following core functionalities were successfully implemented:

Table XII: Implemented Core Functionality

Functionality	What it does
Make Proposal	A user can create a loan proposal specifying terms and interest calculation methods.
Accepting Loan and Funding	Allows the lender to fund the loan of their choice from the proposal explorer by funding the specified amount of ETH in the contract. This advances the loan into the "Funded" state.
Accepting	Allows borrower to accept loan terms and provide the required ETH collateral. This advances the loan into the "Accepted" state.
Taking the Loan	Once the collateral is posted, the borrower can withdraw the loan. This advances the loan into the "Taken" state.
Repayment	Allow borrowers to repay the loan with added-on interest and transfer the collateral back to themselves. The interest will be based on the calculation type specified by the borrower. This advances the loan into the "Repaid" state.
Liquidation	Allows the lender to liquidate the loan if it is not repaid by the due date and receive the collateral. This advances the loan into the "Liquidated state."
Rate Switching	Allows borrowers to switch between floating and fixed interest rates
AI Assistance	Depending on the page the user is on, AI assistance can assist the user by giving general advice or advice based on loan/proposal terms.

Table briefly summarises the core functionalities and their uses.

5.2. Smart Contract Development

Smart Contracts were written in Solidity and compiled with Hardhat in Solidity 0.8.13. The architecture comprises two primary contracts.

With Sepolia and Hardhat, I was able to deploy my LoanFactory through the command line.

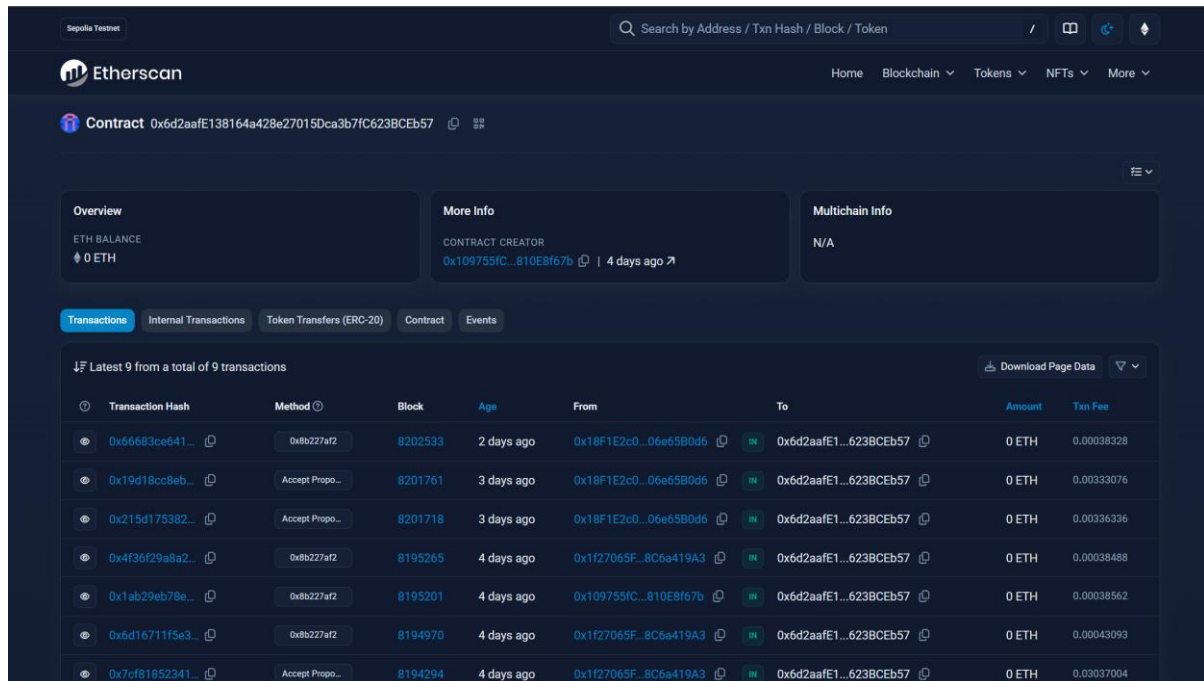
```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat run scripts/deploy-factory.js --network sepolia
Deploying contracts with the accounts: 0x109755fC6Bf1A294D1bCC95C08911a7810E8f67b
LoanFactory deployed to: 0xA3BcaC7FACB62700B1509AE8Cd9Bfb5964A1dA2
```

Figure 5.1: Hardhat command that was used to deploy LoanFactory

Here, functions are numbered, so you can see the full code snippet in Appendix A

5.2.1. LoanFactory.sol

Acts as a factory for the loans by deploying and managing multiple Loan Request contracts as well as managing all the loan proposals and loan proposal acceptances.



The screenshot displays the Etherscan interface for a contract at address 0x6d2aafE138164a428e27015Dca3b7fC623BCEb57. The 'Transactions' tab is selected, showing a list of 9 transactions. The table below represents the data shown in the screenshot.

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x66683ce641...	0x8b227af2	8202533	2 days ago	0x18F1E2c0...06e65B0d6	0x6d2aafE1...623BCEb57	0 ETH	0.00038328
0x19d18cc8eb...	Accept Propo...	8201761	3 days ago	0x18F1E2c0...06e65B0d6	0x6d2aafE1...623BCEb57	0 ETH	0.00333076
0x215d175382...	Accept Propo...	8201718	3 days ago	0x18F1E2c0...06e65B0d6	0x6d2aafE1...623BCEb57	0 ETH	0.00336336
0x4f36f29a8a2...	0x8b227af2	8195265	4 days ago	0x1f27065F...8C6a419A3	0x6d2aafE1...623BCEb57	0 ETH	0.00038488
0x1ab29eb78e...	0x8b227af2	8195201	4 days ago	0x109755fC...810E8f67b	0x6d2aafE1...623BCEb57	0 ETH	0.00038562
0x6d16711f5e3...	0x8b227af2	8194970	4 days ago	0x1f27065F...8C6a419A3	0x6d2aafE1...623BCEb57	0 ETH	0.00043093
0x7cf81852341...	Accept Propo...	8194294	4 days ago	0x1f27065F...8C6a419A3	0x6d2aafE1...623BCEb57	0 ETH	0.03037004

Figure 5.2: Transaction history of a LoanFactory.

Here, I decided to use factory architecture to facilitate the creation of individual smart contracts. The pattern allowed for a modular approach, in which each loan and proposal was isolated and trackable, making it easier to debug. The LoanFactory.sol uses mappings to get loans from lenders and borrowers.

Within the smart contract, I also created a structure for loan proposals to easily handle different loan proposals. When a proposal is accepted, the struct allows us to easily create a LoanRequest we will go on to deploy

```

struct LoanProposal {
    uint256 id;
    address borrower;
    uint256 loanAmount;
    uint256 feeAmount;
    uint256 ethCollateralAmount;
    uint256 repayByTimestamp;
    uint256 fixedRate;
    uint256 floatingRate;
    address oracle;
    bool accepted;
    address acceptedLender;
    uint256 creationTimestamp;
    uint8 interestCalculationType; // 0 for SimpleAPR, 1 for CompoundAPY
}

```

Figure 5.3: LoanProposal struct

Key functions:

- [a] **createProposal():** Stores borrower's loan proposal is based on the conditions they specified. Here, we will use the defined LoanProposal struct to easily create the proposal, and then we will store it in our proposal mapping. We then emit the event so that the front end is notified that the proposal was created.
- [b] **getAllOpenProposals():** Return all proposals that haven't been accepted. This is done by mapping proposals and then confirming whether they have been accepted or not. Those that have not been returned
- [c] **acceptProposal():** Allows the Lender to accept the proposal and create a LoanRequest contract, which gets deployed on the Blockchain. With the terms defined in the LoanProposal, we can now pass them into a LoanRequest smart contract, which we deploy with our LoanFactory. Afterwards, we will add this loan to our lender and borrower mappings and add it to our array of all loans. We then make sure the loan is funded as it is borrowed. We will then go on to emit the event to the front end.
- [d] **getAllLoans():** Return the addresses of all loans which have been deployed on the Blockchain from the loan factory
- [e] **updateUtilizationMetrics():** We will update the utilisation metrics based on the flag given. If we get returned a True, we will add to the totalBorrowed amount; otherwise, we will subtract from it
- [f] **getUtilizationRate():** Will calculate and return the utilisation rate based on total liquidity and the borrowed amount

5.2.2. LoanRequest.sol

Each LoanRequest contract manages the lifecycle of individual loans and makes sure they are secure and only the required users are accessing the Function when needed for security. State management is done through states and valid transitions.

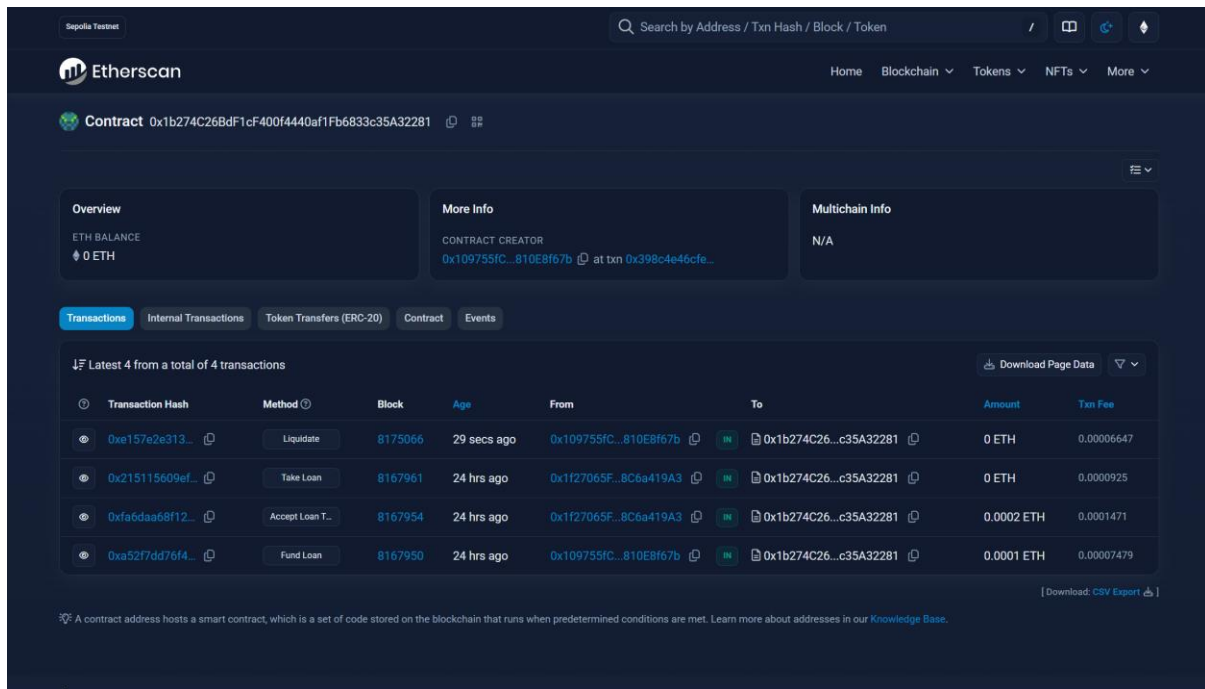


Figure 5.4: Screenshot showing the transaction history of a LoanRequest contract on Etherscan

Key functions:

- [g] **fundLoan():** Allows the Lender to put in the requested amount of ETH if the required statements are fulfilled. It will check that the contract is in the Created state, then check that the lender is funding the loan and make sure that the amount being passed in is correct. Then, the loan will be funded, and the state will be updated to Funded. An event then gets emitted to tell the front end the loan is funded.
- [h] **acceptLoanTerms():** Allows the Borrower to accept the loan terms by providing the requested amount of collateral if the required statements are fulfilled. It will check that the contract is in the funded state, then check that the borrower is accepting the loan and make sure that the amount being passed in is correct. Then, the loan will get the collateral, and the state will be updated to Accepted. An event then gets emitted to tell the front end the loan is accepted.
- [i] **takeLoan():** Allows the Borrower to take their requested loan amount if the required statements are fulfilled. It will check whether the contract is in the Accepted states and whether the borrower is the one taking the amount. Then the borrower can access the funds, and the loan is progressed into the Taken state
- [j] **repay():** Allows the Borrower to repay the loan amount given they have enough time left if the conditions are satisfied. Will check that the borrower is the one accessing it and the loan is in the Taken state, allowing the borrower to repay the loan and move it into the Repaid state
- [k] **liquidate():** Allows the Lender to liquidate the loan to receive the collateral after time has run out. Will check that the lender is the one accessing it and the loan is in the Taken state whilst

the timestamp has passed, allowing the lender to liquidate the loan and move it into the Liquidated state

- [l] **updateRates():** Updates the floating and fixed rates for the contract by getting the most recent Oracle rate and adding the utilisation rate to it
- [m] **switchRateType():** Will allow the borrower to switch to the floating rate if they have the fixed rate enabled and vice versa
- [n] **calculateInterest():** This will calculate the interest the user needs to pay. This is done by checking if the contract ended on a floating rate. If so, rates are updated. Then, depending on the type of interest rate chosen, a calculation will be done to determine what they pay
- [o] **calculateSimpleInterest():** If the user selected APR, this formula (2) will be used in calculateInterest().
- [p] **calculateCompoundInterest():** If the user selected APY, this formula (1) will be used in calculateInterest()

5.3. Frontend Implementation

The front end was built using React.js, TailwindCSS, and Wagmi v2 SoftUI Dashboard. Allowing each view to interact with the smart contracts as needed. This allowed me to create a range of reusable components streamlining the frontend development, allowing for an intuitive user interface through the pages I deliver to the users. In this section, I used the Wagmi hooks mentioned in 4.3.2 to allow for easy connection to the smart contract.

5.3.1 Dashboard.js

- The dashboard can also be seen as the home page, as it is where the user will start and cannot leave unless they have connected to their crypto wallet, such as MetaMask. This was made possible by protecting routes if they haven't connected.
- On this page, we show the Blockchain we are connected to as well as our balance (after the user has connected)
- On this page, we display the ETH price as well as current market conditions, such as the current utilisation rate and floating rate, which will adjust as loans are taken and paid into the system. The prices were fetched using the coingecko api
- Here, we also have our general loan assistant, who can give you general advice and make enquiries about ETH prices.

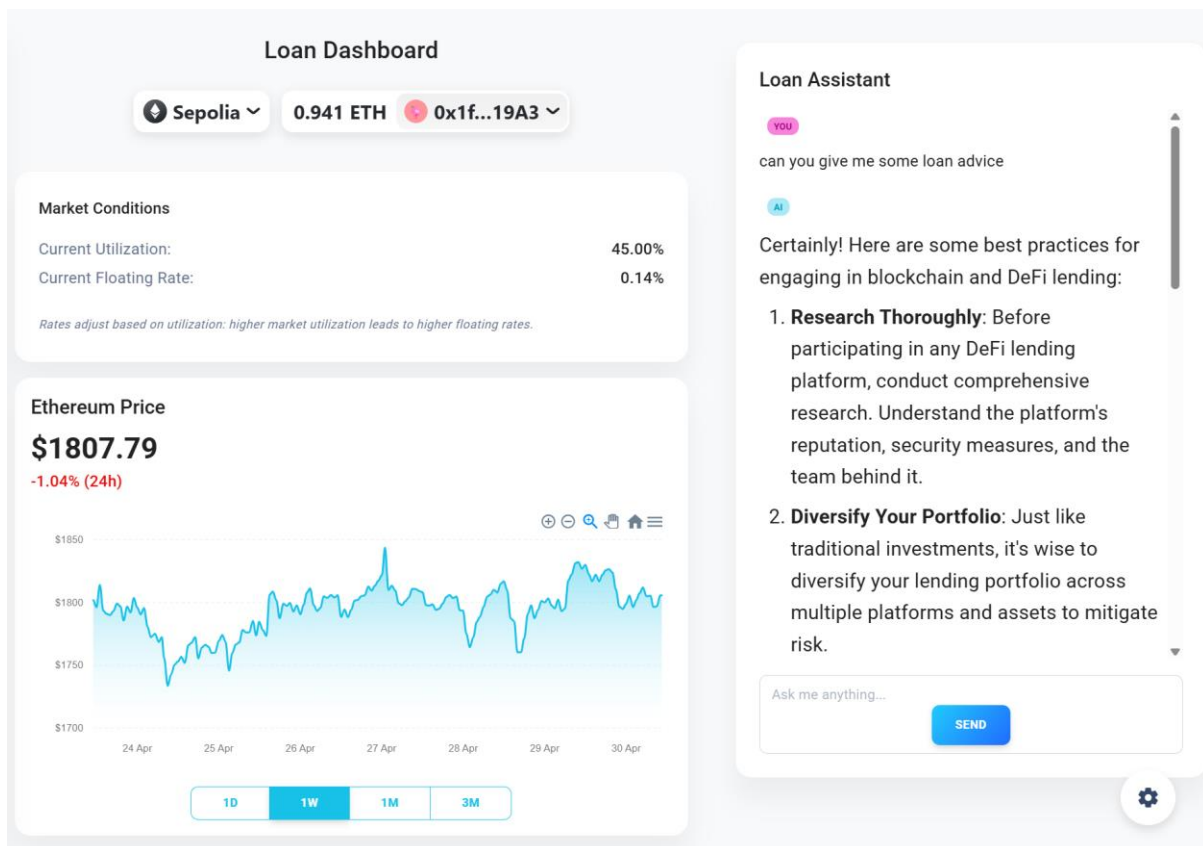


Figure 5.5: Screenshot of the system Dashboard

5.3.2. CreateProposal.js

- Here is where the user makes their proposal. The desired terms will be filled in and then when the button is pressed a LoanProposal structure is made using the conditions that were sent in thanks to the use of Wagmi v2.
- There are conditions here, so proposals are reasonable. For example, if the user tries to put a very low or high amount, they are prompted to use a more reasonable value.
- The assistant will also look at the current filled-out fields and give their opinion on their terms.
- If the fields are not filled, it will define the terms for the user if they are unsure.

Create Proposal

INTEREST CALCULATION TYPE: Simple Interest (APR)

Simple APR applies interest linearly, while Compound APY compounds interest daily

LOAN AMOUNT (ETH)
1

FEE AMOUNT (ETH)
0.1

COLLATERAL (ETH)
5

REPAY BY
07/05/2025 11:57

FIXED RATE (%)
1.14%

FLOATING RATE (%)
0.14%

CURRENT UTILIZATION (%)
45.00%

Higher utilization leads to higher rates

CREATE PROPOSAL

Loan Assistant

Certainly! Let's optimise your blockchain loan proposal to make it fair and attractive for both the borrower and the lender. Here's a structured approach to improving the proposal:

1. Suggestions for Improving Loan Terms

- **Loan Amount & Collateral:**
 - The collateral amount is 5 ETH for a loan of 1 ETH, which is a 500% collateralisation ratio. This is quite high and provides significant security for the lender, but may be burdensome for the borrower. Consider reducing the collateral, if possible, to free up more of your assets, while still maintaining enough to cover the risk for the lender.

GET AI SUGGESTIONS

Figure 5.6: Screenshot of the Create Proposal page

5.3.3. ProposalExplorer.js

- Here is where the user can look through the available proposals. They can then go on to fund the loan they like which will create a LoanFactory smart contract with those desired conditions thanks to Wagmi v2.
- They can get an AI analysis on the proposal to help with their decision based on the details of a proposal they ask for an analysis on.

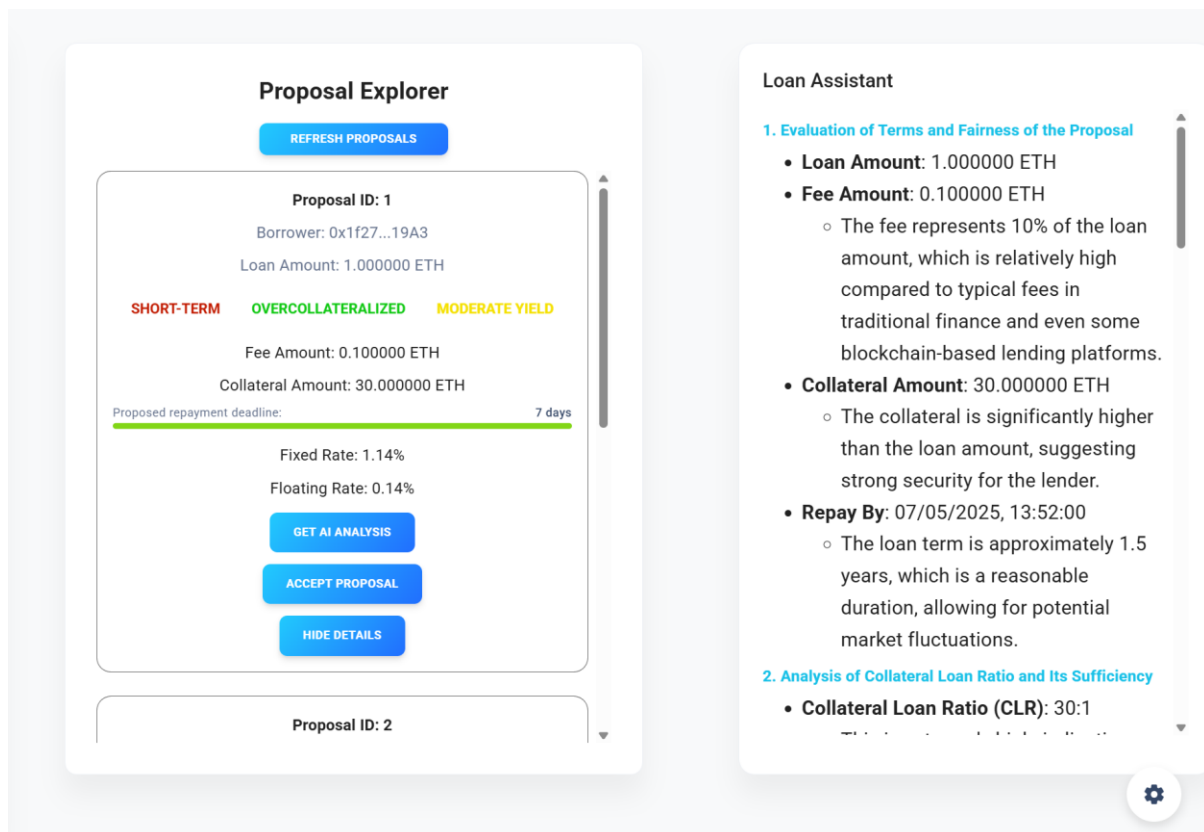


Figure 5.7: Screenshot of the Proposal Explorer

5.3.4. MyLoans.js

- Here is where the user can see the loans they have accepted and are registered as the lender on
- They are able to get an analysis of the loan, helping them understand the terms
- They are also able to liquidate the loan if time runs out allowing them to receive the collateral funds

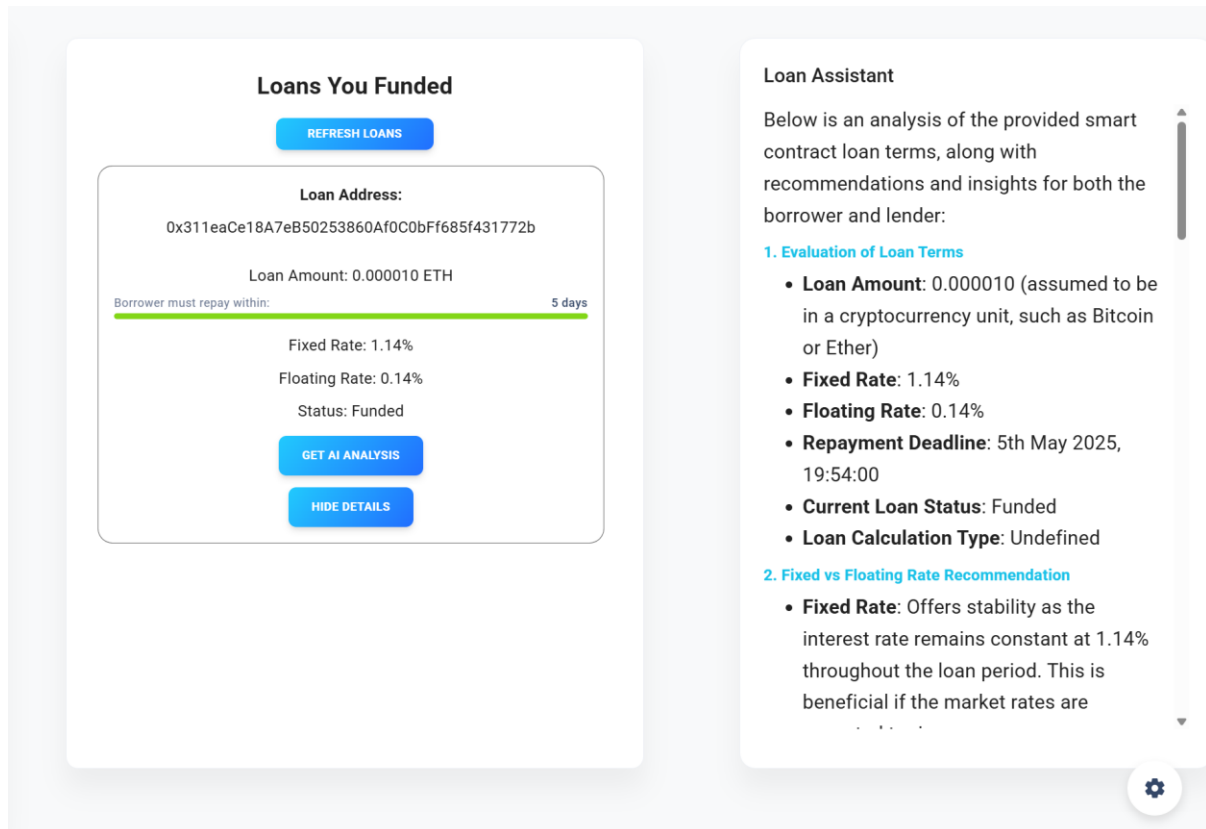


Figure 5.8: Screenshot of the My Loans page

5.3.5. MyBorrowedLoans.js

- Here is where the user can see the loans they have borrowed
- They can get an analysis of the loan, helping them understand the terms
- They are also able to repay the loan, give collateral and take the loan based on the state the loan is in at the time

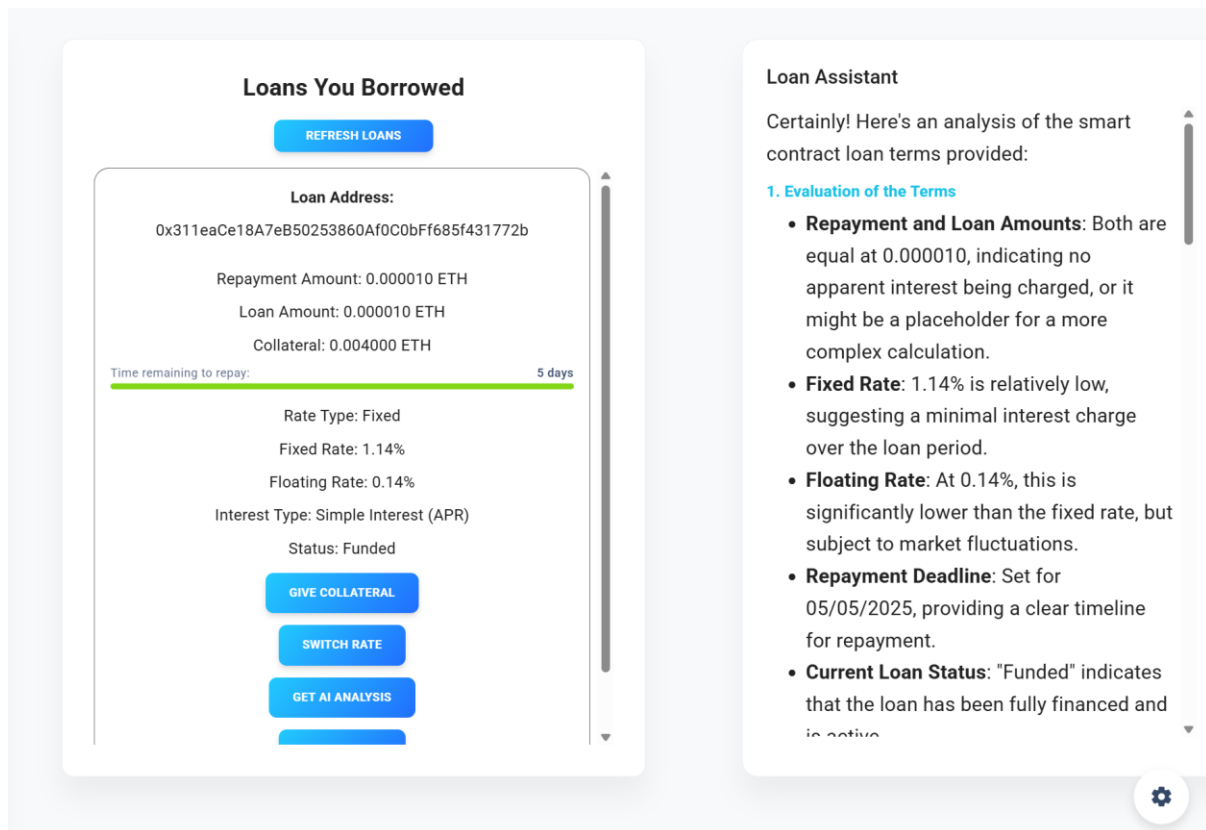


Figure 5.9: Screenshot of the My Borrowed Loans page

5.4. Oracle Implementation

For the Oracle implementation, I used AggregatorV3Interface, an interface that I used to reference the data feed API. This made it much faster to use due to the numerous functions within it.

```
interface AggregatorV3Interface {
    function decimals() external view returns (uint8);

    function description() external view returns (string memory);

    function version() external view returns (uint256);

    // getRoundData and latestRoundData should both raise "No data present"
    // if they do not have data to report, instead of returning unset values
    // which could be misinterpreted as actual reported values.
    function getRoundData(uint80 _roundId)
        external
        view
        returns (
            uint80 roundId,
            int256 answer,
            uint256 startedAt,
            uint256 updatedAt,
            uint80 answeredInRound
        );

    function latestRoundData()
        external
        view
        returns (
            uint80 roundId,
            int256 answer,
            uint256 startedAt,
            uint256 updatedAt,
            uint80 answeredInRound
        );
}
```

Figure 5.10: Code snippet of AggregatorV3Interface.sol

I used the Chainlink USD/ETH price feed for dynamic floating rate computations, and I felt it was the data feed that made the most sense. The Interface allowed borrowers to see the current floating rate.

5.5. AI Assistant Implementation

The AI assistant was integrated into the application as a support tool to improve user engagement and the UX.

The assistant provides understanding guidance to the users, explains technical terms to the users, and evaluates terms. I used GPT-4o to allow for two analysis types: one to evaluate market conditions and one to evaluate loan terms. GPT-4o had allowed for low latency responses embedded in the application. Integration of GPT was all made possible thanks to the OpenAI API.

Depending on the page the user is on, the AI assistant will provide a different service.

Table XIII: Page and AI Assistant use on Page

Page	AI Assistant Use
Dashboard	Users can type in natural language to ask the chatbot for any advice on trading, which will then be outputted.
Create Proposal	Here, you can get an AI analysis on terms you have defined to see any suggestions as well as get what each term means,
Proposal Explorer	Here, you can get a loan proposal broken down to you so that you can see if it is worth funding.
My Loans	Here, the assistant will break down the loan terms and the current state of the loan. Evaluating things like the time left and the state it is in
My Borrowed Loans	Here, the assistant will break down the loan terms and the current state of the loan. Evaluating things like the time left and the state it is in


```

export const helpMakeProposal = async (proposalDetails) => {
  const config = ApiConfig.analysisTypes.loanTerms;

  const prompt = `As a blockchain loan expert, help optimise this loan
proposal:
  - Loan Amount: ${proposalDetails.loanAmount || "Not specified"} ETH
  - Fee Amount: ${proposalDetails.feeAmount || "Not specified"} ETH
  - Collateral Amount: ${proposalDetails.collateral || "Not specified"} ETH
  - Repay By: ${proposalDetails.repayByDateString || "Not specified"}
  - Fixed Rate: ${proposalDetails.fixedRate || "Not specified"}
  - Floating Rate: ${proposalDetails.floatingRate || "Not specified"}

  Provide:
  1. Suggestions for improving these loan terms to be fair and attractive
  2. Recommended collateral-to-loan ratio based on current market conditions
  3. Advice on appropriate fee structure
  4. Optimal repayment timeframe
  5. Explanation of fixed vs floating rate choice for this particular
proposal
  6. Any other recommendations to make this proposal more likely to be
accepted

  If any values are missing, provide general advice for that parameter.
  Format the response in a clear and structured way, using bullet points or
numbered lists where appropriate.
  The rates cannot be adjusted and are deteremined by the market so keep
that in mind.
  Fixed rate stays same throughout the loan period, while floating rate can
change based on market conditions.
  You are talking to the borrower and not the lender so try to make the
proposal as attractive as possible for the lender but also beneficial to the
borrower.
  If fields are not specified, provide general advice for that parameter and
define the field means as some people might not know what a field means such
as collateral.

  Use British Grammar
  `;

  return generateCompletion(prompt, config);
};

```

Figure 5.11. Example Prompt, which has how I engineered my prompt from the Loan Explorer

The message gets sent to our openai.js file, where we will answer based on what type of request was put through.

Table XIV: Prompt Type and What Gets Returned

Prompt Type	What gets returned
analyseLoanTerms	Here, loan terms are taken in, and a breakdown is provided, including an evaluation of terms, risks and benefits.
analyseProposal	Here, proposal terms are taken in, and a breakdown is provided, which includes things like an evaluation of risks to the lender as well as the fairness of the proposal.
helpMakeProposal	Current proposal terms are put in for analysis, and improvements are suggested. For fields that are not filled, the term is defined.
askLoanQuestion	Here, an answer is given to the user based on best practices in DeFi and Blockchain.

Replies then get styled in a nice visual way for the user to look at, which also fits the aesthetic of the rest of the system.

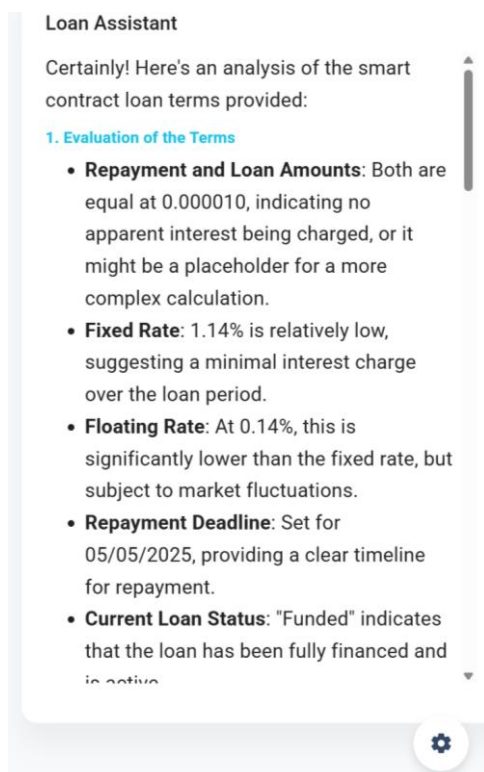


Figure 5.12: Example AI assistant output breaking down terms

5.6. Problems Encountered and Solutions

Table XV: Problems Encountered and Solutions

Problem	Description	Resolution
Incorrect Passing of Values	From the frontend, I was passing incorrect values causing errors	Made sure the frontend logic matched the backend
Math Error	I ran into multiple calculation errors during implementation. In particular, the repayment system had incorrect calculations, which made it so the values would concatenate.	I used the right syntax afterwards while ensuring I had used the correct values for the repayment amount calculation.
Insufficient Funds	I ran into unit test failures, including using insufficient values on deployment. The values led to the borrower balance being too small when testing repayment and mismatches between allowances and requirements.	I made sure to mine some SepoliaETH for a few hours on all accounts using the PoW Faucet
Environment Setup	There was difficulty setting up Ganache and connecting to its local Blockchain.	So, I opted for sepolia as it was much more supported.

Table summarising some of the problems I encountered and how I resolved them

6. Testing and Deployment

6.1. Unit Testing

Thanks to Hardhat, I was easily able to write unit tests for both the smart contracts using Java, which allowed me to run tests without needing to go onto the front end and use ETH. Here, I looked to test key functionality from the backend to make sure the smart contract was secure. Testing aimed to cover most corners, including loan initialising, interest rate management, proposal creation, liquidity management and utilisation rate management

```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat test
```

```
12 passing (1s)
```

Figure 6.1: Command used for testing and tests passing the unit tests

Check Appendix C for full unit testing code and results

6.2. Deployment to Sepolia Testnet

Smart contracts were deployed to the Sepolia Ethereum Testnet using Hardhat using a command. I selected Sepolia due to its compatibility with Chainlink Oracles and its ability to provide a realistic network simulation of the Ethereum Blockchain. I found Sepolia to be much easier to work with overall than other tenets.

Setup and Deployment Steps:

- 1 Create MetaMask accounts and, through the application, add an account to get a second wallet for testing, allowing for the use of two wallets, one that will pose as the borrower and one that will pose as the lender. Afterwards, test the other way around and introduce a third account for more rigorous testing.
- 2 Fund test accounts with SepoliaETH through the use of a faucet. I used Sepolia PoW Faucet to get more test currency to work with. <https://sepolia-faucet.pk910.de/>
- 3 Compile contracts with the npm hardhat command

```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat compile
```

Figure 6.2: Hardhat command for compiling smart contracts

- 4 Deploy the factory using the deployment scripts I had written for the smart contract (can be found in Appendix D) using the npm hardhat command

```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat run scripts/deploy-factory.js --network sepolia
>>
Deploying contracts with the accounts: 0x109755fC6Bf1A294D1bCC95C08911a7810E8f67b
LoanFactory deployed to: 0x81fBF20C8C740c8Cd930EBf80aED13044F8Bc314
```

Figure 6.3: Hardhat command for deploying LoanFactory

5 Confirm deployment of contract on Etherscan

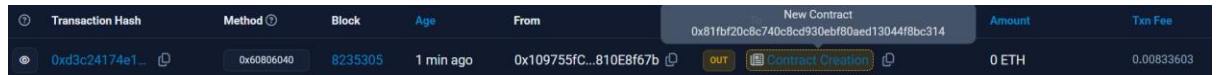


Figure 6.4: Proof of deployment on Etherscan

6 Copy the address into FactoryConfig.js in the front and test

Notes:

- Sepolia Chain ID: 11155111
- Deployment Provider: DRPC/Alchemy
- Verification: Used Hardhat Etherscan plugin

6.3 Integration and End-to-End Testing

Here, I have used two types of tables, one for defining my tests and another for running them and my results

6.3.1 Dashboard Screen

Action	Test	Expected Results
1	Connect Wallet	Prompted to pick a wallet of their choosing
2	Pick MetaMask for the wallet.	Prompted to sign into MetaMask wallet
3	Connect to MetaMask via the extension popup and confirm the extension	Prompted to confirm connection so that now we are signed in and can see our wallet
4	Ensure the displayed Ethereum Price is accurate	Value is reasonably close to the value of ETH
5	Confirm Floating Rate and Utilisation are accurate.	Get a floating rate close in price to the current Sepolia value, and Utilisation is correct for how much is in the smart contract.
6	Ensure the Loan Assistant gives advice information based on what I have asked.	Will return relevant advice in an organised manner

7	If the user closes the tab, make it so they do not have to log back in if it is within a reasonable time.	The user will not be prompted to log in again and can resume where they left off.
8	Allow users to interact with the Ethereum chart to extract the information they want	The user will be able to zoom in on the chart and toggle around.

Action	Method	Met Result	Solution
1	Select the Connect Wallet button	Passed	N/A
2	Pick MetaMask on the option of wallets	Passed	N/A
3	Input the password and confirm when the MetaMask popup appears	Passed	N/A
4	Check the current price when the page is first opened, and then google the current price and inspect that they are close in value.	Passed	N/A
5	Check the price that is returned from Sepolia Chainlink Oracle and make sure the floating rate value is correct. Then, check how much Liquidity and Borrowed funds there are and confirm the Utilisation.	Passed	N/A
6	Ask the assistant a range of questions and check if they are relevant to the scenario.	Passed	N/A
7	Log in, close the tab, then reopen a new one for the website and check if we are still signed in.	Passed	N/A
8	Play with the numerous functions we have displayed on the chart and check that they work as planned.	Passed	N/A

6.3.2. Create Proposal Screen

Action	Test	Expected Results
1	Enter unreasonably large value for Loan Amount	The site will tell them the value cannot pass a certain threshold
2	Enter an unreasonably large value for the Fee Amount	The site will tell them the value cannot pass a certain threshold
3	Enter unreasonably large value for Collateral Amount	The site will tell them the value cannot pass a certain threshold
4	Enter unreasonably small value for Loan Amount	The site will tell them the value cannot go below a certain threshold

5	Enter a value larger than the loan amount for the fee amount	The site will tell the user this isn't allowed as they can only go up to a certain percentage
6	Enter an unreasonably small value for the Collateral Amount	The site will tell them the value cannot go below a certain threshold
7	Enter a DateTime that has already passed	It will not be given as an option
8	Enter a DateTime that is unreasonable, such as a minute	It will tell them they have to set a minimum time ahead
9	Get AI analysis based on results I have filled in	Will evaluate the terms the user has put in and give their opinion on them
10	Get an AI assistant to evaluate each term	Will tell the user what each term is and clearly explain it
11	Make sure the calculation type is correctly passed through to show the right one.	The type will be shown on the Borrower's My Borrowed Loans screen for the loan.
12	Make sure interest values are correct.	The values will align with the dashboard, and the fixed rate will be 1% above the floating.
13	A proposal is created when Create Proposal is pressed	The proposal will appear on the proposal explorer for any user who is not the borrower

Action	Method	Met Result	Solution
1	Will try to input 1000 ETH	Pass	N/A
2	Will try to input a value larger than the loan amount	Pass	N/A
3	Will try to input 5000 ETH	Fail	I had restricted collateral to 500 ETH as I had a loan amount restricted to 100 ETH.
4	Will try to input 0.00001 ETH	Pass	N/A
5	Will try to input a value larger than the loan amount for the fee amount	Pass	N/A
6	Will try to input 0.00005 ETH	Pass	N/A
7	Will try to set the date back to a week ago	Pass	N/A
8	Will try to set the date to a few minutes into the future	Pass	N/A

9	Will input a range of values into the proposal form and then press "Get AI Suggestions."	Pass	N/A
10	Will leave the form empty, then press "Get AI Suggestions."	Pass	N/A
11	Will check that the calculation type is displayed when the loan is accepted	Pass	N/A
12	Will check values align with the dashboard as those were validated	Pass	N/A
13	Will check Proposal Explorer from a different account to see that the proposal was created with the specified conditions	Pass	N/A

6.3.3. Proposal Explorer Screen

Action	Test	Expected Results
1	Check that refresh proposals work correctly.	Results will show all the current proposals after the refresh.
2	Check that the "SHOW DETAILS" button works correctly	Will open the proposal fully so that we can view all the proposal details
3	Check that the "GET AI ANALYSIS" button works correctly	Will return an AI analysis of the proposal on the Loan Assistant panel, breaking down the proposal and if it is worth accepting
4	Check that the "ACCEPT PROPOSAL" button works correctly	This will allow the user to fund the proposal and create a LoanRequest.sol smart contract for their loan that is on Etherscan
5	Check that the "HIDE DETAILS" button works correctly	Will minimise the details back to the minimalist view
6	Check that tags are assigned correctly to each loan	Will show appropriate tags based on the loan terms
7	Ensure the user's proposal doesn't appear.	Can see all proposals but their own
8	Ensure only the lender with sufficient funds can accept the loan	Will have an alert saying that they do not have sufficient funds

Test	Method	Met Result	Solution
1	Will create a proposal while the user is on the page, then refresh it to see if the new proposal appears.	Pass	N/A
2	Will press the "SHOW DETAILS" button	Pass	N/A
3	Will press the "GET AI ANALYSIS" button	Pass	N/A
4	Will press the "ACCEPT PROPOSAL" button	Pass	N/A
5	Will press the "HIDE DETAILS" button	Pass	N/A
6	Will create proposals quite different in properties and check that the relevant tag is assigned	Pass	N/A
7	Make a proposal, then go to the Explorer and check if it is there	Pass	N/A
8	Will create a large loan the user cannot fund and try accepting it	Pass	N/A

6.3.4. My Loans Screen

Action	Test	Expected Results
1	Check that "REFRESH PROPOSALS" works correctly	Results will show all the currently funded loans after the refreshed
2	Check that the "SHOW DETAILS" button works correctly	Will open the loan fully so that we can view all the loan details
3	Check that the "LIQUIDATE LOAN" button works correctly	Will transfer collateral to the lender only when the loan has expired
4	Check that the "GET AI ANALYSIS" button works correctly	Will return an AI analysis of the loan on the Loan Assistant panel, breaking down the loan and giving a detailed description of its current situation

5	Check that the "HIDE DETAILS" button works correctly	Will minimise the details back to the minimalist view
6	Check that the status is shown correctly	It will show the relevant status depending on what state the loan is in

Test	Method	Met Result	Solution
1	Will accept a loan request while the user is on the page, then refresh it to see if the newly funded loan appears.	Pass	N/A
2	Will press the "SHOW DETAILS" button	Pass	N/A
3	Will wait for the loan to expire, then for the "LIQUIDATE LOAN" button to appear, and then press it.	Pass	N/A
4	Will press the "GET AI ANALYSIS" button	Pass	N/A
5	Will press the "HIDE DETAILS" button	Pass	N/A
6	Will give the collateral from the Borrower account, then use the refresh from the lender side to see if the status has been updated	Pass	N/A

6.3.5. My Borrowed Loans Screen

Action	Test	Expected Results
1	Check that "REFRESH PROPOSALS" works correctly	Results will show all the currently funded loans after refreshed
2	Check that the "SHOW DETAILS" button works correctly	Will open the loan fully so that we can view all the loan details
3	Check that the "GIVE COLLATERAL" button works correctly	Will make the Borrower deposit the prescribed collateral into the smart contract only appears when the loan has reached the funded state

4	Check that the "TAKE LOAN" button works correctly	This will allow the borrower to take the required loan amount from the smart contract, which only appears when the loan has reached the accepted state.
5	Check that the "REPAY" button works correctly.	Will allow the borrower to repay the loan to the smart contract, returning their collateral to them only appears when the loan has reached the taken state
6	Check that the "GET AI ANALYSIS" button works correctly	Will return an AI analysis of the loan on the Loan Assistant panel, breaking down the loan giving a detailed description of its current situation
7	Check that the "HIDE DETAILS" button works correctly	Will minimise the details back to the minimalist view
8	Check that the status is shown correctly	It will show the relevant status depending on what state the loan is in

Test	Method	Met Result	Solution
1	We will accept a loan request while the user is on the page, then refresh it to see if the newly funded loan appears.	Pass	N/A
2	Will press the "SHOW DETAILS" button	Pass	N/A
3	Will press the "GIVE COLLATERAL" button when the loan has reached the funded state	Pass	N/A
4	Will press the "TAKE LOAN" button when the loan has reached the accepted state	Pass	N/A
5	Will press the "REPAY" button when the loan has reached the taken-state	Pass	N/A
6	Will press the "GET AI ANALYSIS" button	Pass	N/A
7	Will press the "HIDE DETAILS" button	Pass	N/A
8	Will give the liquidate a loan from the lender account then use the refresh from the borrower account to see if the status has updated	Pass	N/A

6.3.6. Navbar

Action	Test	Expected Results
--------	------	------------------

1	Check that the user can navigate to the pages of choice	Take the user to the selected page
---	---	------------------------------------

Test	Method	Met Result	Solution
1	Will press all the buttons and check that it directs us to the right page	Pass	N/A

7. Results and Evaluation

This section will discuss and critically evaluate the functionality, performance, user experience and security of the DeFi platform.

7.1. Functional Evaluation

Here, we will use 3 points to critically evaluate our functionality:

1. **Is the Function working as designed?**
2. **Does the Function ensure a smooth transition to the next state/stage?**
3. **Were the Function's edge cases successfully handled?**

Make Proposal - Here, a user creates a proposal by specifying the terms they would like. They will then use the create button to create a LoanProposal struct that is stored in the LoanFactory.sol smart contract. The proposal will then be in the Proposal Explorer view.

1. The Function successfully did all these, and we were able to confirm this in Etherscan.
2. The Function had confirmation that the proposal was created, showing that it had been done clearly.
3. I had run edge case testing in the previous section, and the proposal only allowed for valid terms to be put through.

Accepting Proposal and Funding - Here, a user browses the Proposal Explorer. If they find one they like, they can accept the proposal. By accepting the proposal, they are funding it and creating a LoanRequest.sol smart contract.

1. The Function successfully allowed the user to accept the loan and be registered as the lender on the smart contract created.
2. It showed the user the loan has been created but takes a bit of time to disappear, so we could work on speeding it up so the proposal disappears from the view.
3. I have run edge case testing to make sure only lenders with sufficient funds can fund the loan. This proved to be beneficial in making sure loans were created properly, avoiding later errors.

Giving the collateral for the loan - Once the user's proposal has been accepted, they give collateral to the smart contract so that they can take the loan out.

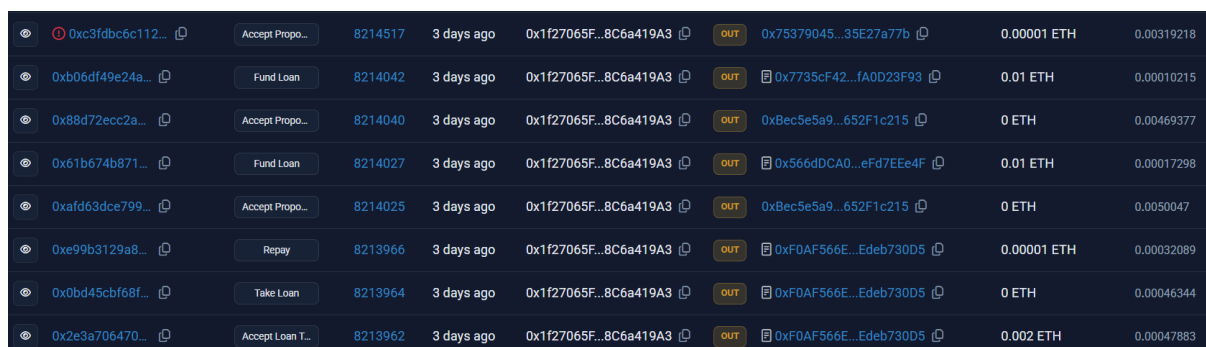
1. The Function allowed the borrower to give collateral for the loan, moving the contract into a state where they take the loan.
2. There is confirmation that the transaction went through on MetaMask from the extension, as well as changing the state of the loan into the Accepted state so they can take out the loan.
3. I ran edge cases so that they were notified if they had insufficient funds to give in the collateral.

Taking the Loan - Once the user has given the collateral, they can take out the loan, transferring the funds into their crypto app, such as MetaMask.

1. The user was able to easily take out the loan as the contract progressed to a stage where that was an option.
2. There is confirmation that the transaction went through on MetaMask from the extension, as well as changing the state of the loan into the Taken state and making it so they can repay the loan.
3. I confirmed that the transaction went through to the right account on MetaMask.

Repayment - Once the user has taken out the loan is ready to pay it back whilst the deadline has not been passed. They should be able to pay back the loan as well as get their collateral back to pay successfully on time.

1. The user was easily able to repay the loan to the contract once the loan was in that stage. This transferred the loan amount to the lender, returned collateral to the borrower and made sure the borrower paid the loan with added-on interest.
2. There is confirmation this has occurred in MetaMask, and the loan is progressing to the Repaid state.
3. I made the user aware that they had insufficient funds if they tried repaying it, given they did not have enough funds in their account.

A screenshot of the Etherscan interface showing a list of transactions. Each row represents a transaction with columns for the transaction icon, a truncated address, a button label (e.g., 'Accept Propo...', 'Fund Loan', 'Repay', 'Take Loan', 'Accept Loan T...'), a transaction ID (e.g., 8214517, 8214042, 8214040, 8214027, 8214025, 8213966, 8213964, 8213962), the time '3 days ago', the 'to' address (0x1f27065F...8C6a419A3), a status 'OUT' in a yellow box, the 'from' address (e.g., 0x75379045...35E27a77b, 0x7735cf42...fA0D23F93, 0xBec5e5a9...652F1c215, 0x566dDCA0...eFd7EEe4F, 0xBec5e5a9...652F1c215, 0xF0AF566E...Edeb730D5, 0xF0AF566E...Edeb730D5), the amount in ETH (0.00001 ETH, 0.01 ETH, 0 ETH, 0.01 ETH, 0 ETH, 0.00001 ETH, 0 ETH, 0.002 ETH), and a gas fee (e.g., 0.00319218, 0.00010215, 0.00469377, 0.00017298, 0.0050047, 0.00032089, 0.00046344, 0.00047883).

🔍	0xc3fdb6c6c112...	Accept Propo...	8214517	3 days ago	0x1f27065F...8C6a419A3	OUT	0x75379045...35E27a77b	0.00001 ETH	0.00319218
🔍	0xb06df49e24a...	Fund Loan	8214042	3 days ago	0x1f27065F...8C6a419A3	OUT	0x7735cf42...fA0D23F93	0.01 ETH	0.00010215
🔍	0x88d72ecc2a...	Accept Propo...	8214040	3 days ago	0x1f27065F...8C6a419A3	OUT	0xBec5e5a9...652F1c215	0 ETH	0.00469377
🔍	0x61b674b871...	Fund Loan	8214027	3 days ago	0x1f27065F...8C6a419A3	OUT	0x566dDCA0...eFd7EEe4F	0.01 ETH	0.00017298
🔍	0xafd63dce799...	Accept Propo...	8214025	3 days ago	0x1f27065F...8C6a419A3	OUT	0xBec5e5a9...652F1c215	0 ETH	0.0050047
🔍	0xe99b3129a8...	Repay	8213966	3 days ago	0x1f27065F...8C6a419A3	OUT	0xF0AF566E...Edeb730D5	0.00001 ETH	0.00032089
🔍	0x0bd45cbf68f...	Take Loan	8213964	3 days ago	0x1f27065F...8C6a419A3	OUT	0xF0AF566E...Edeb730D5	0 ETH	0.00046344
🔍	0x2e3a706470...	Accept Loan T...	8213962	3 days ago	0x1f27065F...8C6a419A3	OUT	0xF0AF566E...Edeb730D5	0.002 ETH	0.00047883

Figure 7.1: Confirmation of transactions on Etherscan

Liquidation - Once the Borrower has taken out the loan and has not paid it back whilst the deadline has passed, the user should be able to extract the collateral into their wallet

1. The lender can liquidate and get the collateral if the deadline pass
2. The contract ends up in a Liquidated state, showing the users that the contract has finished and the lender has now received the collateral
3. The lender is aware of the transaction through MetaMask

Rate Switching - The Borrower shall be able to change the rate type between floating and fixed whilst the loan is active. Once the loan expires, the rate type that was activated will be used to calculate the repayment amount for the user

1. The borrower had updated the smart contract by using the switch rate button, allowing for a different type to be active
2. This was smoothly shown through MetaMask and the contract, which said the rate they had switched to. Both rates were also shown at the time
3. The button had initially still appeared after the deadline had passed, allowing them to manipulate the system. I had fixed this, so now they cannot

Calculation Type - The calculation type between APR and APY that was specified in the proposal creation will be used to calculate the interest amount that will be paid to the user

1. At the time of creating the proposal, the borrower would have selected a calculation type at the end of the loan tenure. The type is used to calculate the final repayment amount
2. The user can see the type they have chosen, and it is reflected in the final amount that they pay to the contract
3. I made sure there was no way to change it and confirmed that my backend formulas were correct, which were based on formulas for (1) APY, (2) APR and (3) Utilisation

AI Assistance – Depending on the page the user is on, the assistant will be able to provide guidance, give general advice and define or break a set of terms to the user when necessary

1. The AI assistant provided relevant information based on the page the user was on and the information that was fed into it
2. The returned statement was clearly formatted for the user in a way they could understand
3. I had tried to use prompts that were completely irrelevant, and it will talk about DeFi either way. I had also tried feeding into it crazy numbers, but despite this, a reasonable response had been returned

Contract Deployment – The LoanFactory.sol smart contract should be able to deploy a LoanRequest.sol based on the values passed in

1. The contract succeeded in creating the LoanRequest, as we can see the transaction on Etherscan as well as all the transactions that occurred in the LoanRequest throughout it
2. This part wasn't shown to the user as it would have been confusing to most
3. I made sure only reasonable attributes would be created through all my restrictions on the Create Proposal page

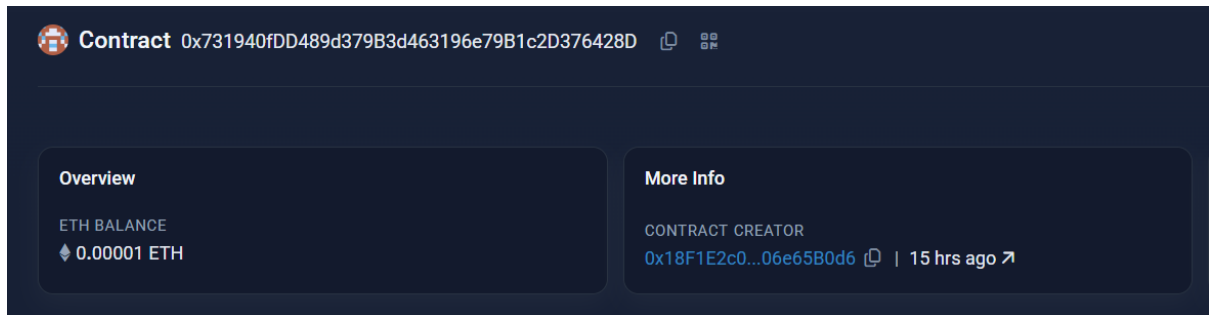


Figure 7.2: A loan that was freshly deployed with the lender's collateral

7.2. Performance and Optimisation

I have tried my best to ensure the best possible gas use and use of resources for my smart contract.

7.2.1. Gas Efficiency

Gas usage was optimised throughout the smart contract logic. Here are the following average gas fees for some of my most frequently used functions:

Solidity and Network Configuration					
Solidity: 0.8.28	Optim: true	Runs: 200	viaIR: false	Block: 30,000,000 gas	
Methods					
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
LoanFactory	-	-	-	-	-
acceptProposal	-	-	1,520,389	1	-
addLiquidity	-	-	26,393	2	-
createProposal	253,941	256,741	254,501	5	-
removeLiquidity	-	-	35,893	1	-
updateUtilizationMetrics	-	-	26,928	1	-
LoanRequest	-	-	-	-	-
acceptLoanTerms	-	-	91,728	6	-
switchRateType	28,632	28,637	28,635	2	-
takeLoan	-	-	72,047	6	-
Deployments	-	-	-	% of limit	-
LoanFactory	-	-	2,612,639	8.7 %	-

Figure 7.3: Screenshot of gas fees on functions used on the system

Average gas was worked out by changing my hardhat.config.js file so that I could get gas used from it and put it into a text file

Although contract deployments weren't as efficient as I wanted, I made sure to actually make sure the functions themselves were efficient to ensure a gas-friendly system. Measures I have taken include:

- Reducing storage variable updates
- Trying to use efficient data structures such as mappings and minimal arrays

```
mapping(uint256 => LoanProposal) public proposals;  
mapping(uint256 => address) public proposalToAddress;
```

Figure 7.4: Some of the mappings used

- Tried to use the most minimal amount of contract calls I could so that we wouldn't be unnecessarily wasting gas

```
function fundLoan() external payable{  
    require(state == LoanState.Created, "Loan is not in Created state");  
    require(msg.sender == lender || msg.sender == factory , "Only lender  
can fund the loan");  
    require(msg.value == loanAmount, "Incorrect loan amount");  
  
    lender = payable(msg.sender);  
    state = LoanState.Funded;  
  
    emit LoanFunded(msg.sender, loanAmount);  
}
```

Figure 7.5: fundLoan solidity function

7.2.2. System Responsiveness

I had also done a lot of work to keep wait times short by trying to implement a highly responsive system. Some wait times of some of our most used functions are shown below.

Transaction	Wait Time (Approximation)
fundLoan	6 seconds
switchRate	4 seconds
createProposal	7 seconds

Functions and wait times for each one

On Sepolia, most transactions were confirmed to be the same amount, with loan acceptance being the most time-consuming at 20 seconds, as a whole smart contract was being deployed.

Using hooks like `useWaitForTransactionReceipt` and `useReadContract` allowed the UI to update 2-4 seconds post confirmation.

```
const { isLoading: isWaiting, isSuccess } = useWaitForTransactionReceipt({
  hash: txHash,
  onSuccess: (receipt) => {
    console.log("Transaction receipt:", receipt);
  },
  onError: (error) => {
    console.error("Error waiting for transaction receipt:", error);
  },
});
```

Figure 7.6: Use of `useWaitForTransactionReceipt`

7.2.3. AI Assistant Responsiveness

The platform's AI assistant, which was on all the pages, directly contributes to the user experience, so it is crucial that it is as efficient as possible.

Page	Average Response Time
Dashboard	5 seconds
Create Proposal	7 seconds
Proposal Explorer	7 seconds
My Loans	7 seconds
My Borrowed Loans	7 seconds

Summarisation of wait times for AI assistant on each page

Loan Assistant

1. Evaluation of Terms and Fairness

- **Loan Amount:** 1 ETH
- **Fee Amount:** 0.1 ETH
 - The fee amounts to 10% of the loan, which is relatively high for a loan of this size and may impact the overall cost for the borrower.
- **Collateral Amount:** 30 ETH
 - This is exceptionally high collateral for a 1 ETH loan, suggesting a very conservative approach towards risk.
- **Repay By Date:** 07/05/2025
 - The repayment period gives ample time for the borrower to repay, assuming market conditions remain stable.

Figure 7.7: Example output of loan assistant

7.3. User Experience and Interface

I made sure to put a lot of emphasis on usability clarity and ease of interaction.

7.3.1. Wallet Integration

Using Rainbowkit allowed for an intuitive and visually appealing wallet connection flow, simplifying the wallet connection process. With MetaMask, I was able to allow seamless user onboarding with clear popups throughout the transactions.

7.3.2. Frontend Clarity and Usability

The frontend was also designed to enhance the user's confidence through using the system due to the intuitive nature of it boasting a range of features such as:

- Fetching real-time ETH prices, loan statuses, loan rates
- A proposal explorer with keywords showing the user what qualities each loan has, such as being long-term or highly collateralised.
- Loan management interfaces provide clear actions for repayments and rate-switching interactions.

I also made sure to use Soft UI Dashboard React to allow for a visually appealing user experience.

7.3.3. User Feedback and Loading States

I also tried to use clear indicators of progress, such as a progress bar for the time left of the contract whenever a contract/proposal was enlarged. I also provided immediate visual feedback to the user.

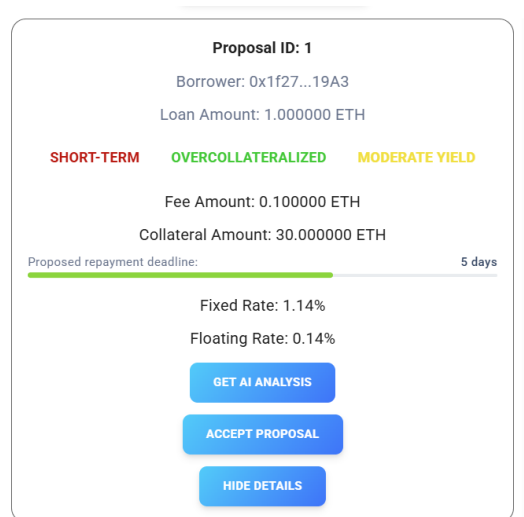


Figure 7.8: Example proposal loan

7.4. Security and Reliability

I worked to ensure that the security and reliability of the smart contract implementation was a strong point.

7.4.1. Smart Contract Security

I had heavily implemented Solidity's required statements to ensure proper validation of loan state interactions fund management and reduce the chances of the system being broken.

```
function liquidate() external {
    require(state == LoanState.Taken, "Loan is not in Taken state");
    require(block.timestamp > repayByTimestamp, "Repayment deadline has not passed");
    require(msg.sender == lender, "Only lender can liquidate the loan");

    state = LoanState.Liquidated;
    lender.transfer(ethCollateralAmount);
    emit LoanLiquidated(borrower, lender, ethCollateralAmount);
}
```

Figure 7.9: use of required in liquidation functions

I also made sure to use careful state management and variable ordering to vulnerabilities and only make it so the users can only access the smart contract at a certain time

7.4.2. Oracle and Frontend Reliability

Chainlink ETH/USD price proved stable, providing accurate market data for interest rate calculations.

I also made sure to implement refresh buttons for freshness to prevent incorrect data

I also made sure to implement error checking on the whole stack to ensure that the system does not fail at any point, vastly improving the system's reliability

8. Limitations and Challenges

Despite fully implementing the system, it also comes with its flaws. In this section, I will highlight the flaws of the system and talk about how it has impacted the overall design

8.1. Technical and Functional Limitations

Collateral Pricing Model: - The collateral system comes with the flaw that you can only provide collateral in ETH. This negatively impacted the system as it reduced flexibility for borrowers who would like to use other tokens. This means that if a user would like to use a system that would allow them to transfer their tokens to ETH through another application, it can cause them to lose money through gas fees, all before being able to use the system. The reason it was implemented like this was to simplify the design of the project

No automated liquidation trigger - For the loan to be fully liquidated, the lender has to manually take out the collateral once the deadline has passed. This is inconvenient and does not align with the streamlined user experience I had aimed for. The reason it was implemented like this was to allow for more focus on the smart contract, but this is definitely something I would like to implement in the future.

Static Chainlink feed usage – The platform only uses the Chainlink ETH/USD Oracle and doesn't utilise other available oracles. This means that there could be a chance of error when calculating our floating rate if the oracle is experiencing downtime.

No use of database – All data is either stored on a chain or derived from the frontend state. This is an issue as no data, such as AI chat history, is retained beyond the session. This is due to the project being decentralised by nature, but it would definitely be something worth implementing in the future.

AI assistant Not On-Chain – The AI assistant is frontend-based and does not interact with the chain beyond what is fetched from the frontend. This limits the depth of the questions we can ask the assistant. The AI assistant wasn't the main goal of the project, but it still provides great value to the user in its current version.

8.2. Project Challenges and Obstacles

Solidity Interest Logic Errors – Early on in development, my calculateInterest Function returned incorrect values. This was due to using repayment deadlines rather than the time the loan was taken.

Truffle to Hardhat Migration – Initial testing was done using Truffle, but eventually, it was time to migrate to Hardhat to support proper contract deployments. I refactored my deployment scripts and created a Hardhat config file. The change proved to be beneficial in the long term.

Flow coordination – Ran into issues where the wrong buttons would appear at the wrong times, interrupting the intended contract flow and causing the smart contract to be prone to errors. I used stronger validation and a lot of conditions to ensure that only the right buttons appeared, removing buttons that would cause errors.

Limited Initial Knowledge of Solidity and React – Going into the project, I had no prior knowledge of Solidity or React, which caused me to spend some time researching and practising. Eventually, I felt confident enough to get implemented, and I felt my skill in these languages grow as I went on, making it very enjoyable.

9. Conclusion

Here, I will conclude the project by talking about what I have achieved through this project, what I have learned from this project and future enhancements I can add to the system in the future.

9.1. Summary of Achievements

The project successfully delivered a fully functional DeFi application on the Ethereum Sepolia Blockchain, where I was able to add my own touch to the rate-switching functionality. I successfully allowed borrowers and lenders to interact through the smart contract features. This is only some of the successes achieved through the project. Key achievements include:

- Secure and Modular smart contract system through my LoanFactory and LoanRequest smart contracts.
- Full frontend interface built using SoftUI React, TailwindCSS, Wagmi v2 and Rainbowkit, which is easy to use and also visually appealing
- Switching between floating and fixed-rate switching
- Real-time Oracle integration for ETH/USD price data
- Metamask integration so that users can use their existing wallets or sign up through an existing secure and trusted service
- The option to choose what type of interest calculations the user uses between APR and APY
- A dynamic interest rate calculation system enabled by the use of utilisation calculations
- An embedded AI assistant offering guidance in different ways through evaluation of terms and general advice
- View current ETH prices so that users can get an idea of how much ETH is at the point they are using it.

All contracts were deployed and tested on Sepolia with Metamask on the front end, allowing for complete lifecycle management. The project followed good practices with unit tests, error handling and gas efficiency considerations.

9.2. Learning and Insights

The project was challenging in nature due to it being full stack and decentralised. Having initially come in with no knowledge of Solidity and React, I challenged myself to rapidly acquire the skillset needed for Blockchain Development, Frontend Development, smart contract testing and wallet integration.

Key learning outcomes include:

- Writing secure modular Solidity contracts
- Managing a range of React interfaces with Wagmi hooks

- Integrating real-time oracles with Chainlink
- Knowledge of key practices for managing the efficiency of a contract
- Structuring a Dapp so that it is usable and reliable

9.3. Potential Additions

I can include a range of enhancements to enhance the system.

Multi-asset Collateral Support - Introduce other tokens as collateral so that there is more flexibility. Other tokens can include LINK.

Automated liquidation trigger – Make it so that liquidity is automatically transferred to the user when the repayment date passes. This could be done by tracking from the front or maybe using a Chainlink Automaton.

Multiple Oracles – Integrate multiple oracles into the system so that we can have something to fall to during downtime

Use of database – I can introduce an off-chain backend with PostgreSQL to log historical data, enable analytics, or maybe even create a user system unique to my program.

AI assistant Expansion– Make it so the AI assistant can interact with the chain to provide better information.

Ethereum Blockchain use – I used Sepolia because it was adequate enough to demonstrate the system capabilities, but I hadn't deployed it on the actual Ethereum Blockchain. The system was designed so that it can be easily integrated in the future and would provide a range of benefits.

9.4 Closing Thoughts

The project was a welcoming challenge for me as it equipped me with a range of skills relevant to the growing world of DeFi and allowed me to apply my mathematical background to my coding. Having a designed and deployable blockchain has definitely been a valuable experience. The project demonstrated how DeFi can be used to solve meaningful problems in Blockchain and enhance user transparency and flexibility. I am leaving this project confident in my ability to build a scalable advanced system in the DeFi space.

Appendix

Appendix A: Code snippets

LoanFactory.sol

[a] createProposal()

```
function createProposal(
    uint256 _loanAmount,
    uint256 _feeAmount,
    uint256 _ethCollateralAmount,
    uint256 _repayByTimestamp,
    uint256 _fixedRate,
    uint256 _floatingRate,
    address _oracle,
    uint8 _interestCalculationType // 0 for SimpleAPR, 1 for CompoundAPY
) external {
    proposals[nextProposalId] = LoanProposal({
        id: nextProposalId,
        borrower: msg.sender,
        loanAmount: _loanAmount,
        feeAmount: _feeAmount,
        ethCollateralAmount: _ethCollateralAmount,
        repayByTimestamp: _repayByTimestamp,
        fixedRate: _fixedRate,
        floatingRate: _floatingRate,
        oracle: _oracle,
        accepted: false,
        acceptedLender: address(0),
        creationTimestamp: block.timestamp,
        interestCalculationType: _interestCalculationType // Default to
SimpleAPR
    });
    emit LoanProposalCreated(nextProposalId, msg.sender);
    nextProposalId++;
}
```

[b] getAllOpenProposals():

```
function getAllOpenProposals() external view returns (LoanProposal[]
memory) {
    uint256 openProposalsCount = 0;
    for (uint256 i = 0; i < nextProposalId; i++) {
        if (!proposals[i].accepted) {
            openProposalsCount++;
        }
    }

    LoanProposal[] memory openProposals = new
LoanProposal[](openProposalsCount);
    uint256 index = 0;
    for (uint256 i = 0; i < nextProposalId; i++) {
        if (!proposals[i].accepted) {
            openProposals[index] = proposals[i];
            index++;
        }
    }

    return openProposals;
}
```


[c] acceptProposal():.

```
function acceptProposal(uint256 _proposalId) external payable returns
(address) {
    LoanProposal storage proposal = proposals[_proposalId];
    require(!proposal.accepted, "Proposal already accepted");
    require(msg.value == proposal.loanAmount, "Incorrect ETH amount
sent");

    proposal.accepted = true;
    proposal.acceptedLender = msg.sender;

    // Create a new LoanRequest contract
    LoanRequest newLoan = new LoanRequest(
        proposal.loanAmount,
        proposal.feeAmount,
        proposal.ethCollateralAmount,
        proposal.repayByTimestamp,
        proposal.fixedRate,
        proposal.floatingRate,
        proposal.oracle,
        address(this),
        msg.sender, // Pass the original sender as lender
        proposal.creationTimestamp, // Pass the creation timestamp to the
LoanRequest contract
        uint8(proposal.interestCalculationType) // Pass the interest
calculation type
    );

    // Store the loan address in the mapping and array
    loansByLender[msg.sender].push(address(newLoan));
    loansByBorrower[proposal.borrower].push(address(newLoan));
    allLoans.push(address(newLoan));

    proposalToAddress[_proposalId] = address(newLoan);

    LoanRequest(newLoan).fundLoan{value: msg.value}();

    emit LoanProposalAccepted(_proposalId, msg.sender, address(newLoan));
    return address(newLoan);
}
```

[d] getAllLoans():

```
function getAllLoans() external view returns (address[] memory) {  
    return allLoans;  
}
```

[e] updateUtilizationRate():

```
function updateUtilizationMetrics(uint256 amountBorrowed, bool isIncrease)  
public {  
    if (isIncrease) {  
        totalBorrowed += amountBorrowed;  
    } else {  
        if (amountBorrowed > totalBorrowed) {  
            totalBorrowed = 0;  
        } else {  
            totalBorrowed -= amountBorrowed;  
        }  
    }  
}
```

[f] getUtilizationRate():

```
function getUtilizationRate() public view returns (uint256) {  
    if (totalLiquidity == 0) return 0;  
    return (totalBorrowed * 10000) / totalLiquidity; // Returns  
utilization rate as a percentage * 100  
}
```

LoanRequest.sol

[g] fundLoan(): Allows the Borrower to put in the requested amount of ETH

```
function fundLoan() external payable{
    require(state == LoanState.Created, "Loan is not in Created state");
    require(msg.sender == lender || msg.sender == factory, "Only lender
can fund the loan");
    require(msg.value == loanAmount, "Incorrect loan amount");

    lender = payable(msg.sender);
    state = LoanState.Funded;

    emit LoanFunded(msg.sender, loanAmount);
}
```

[h] acceptLoanTerms(): Allows the Borrower to accept the loan terms by providing the requested amount of collateral

```
function acceptLoanTerms() external payable {
    require(state == LoanState.Funded, "Loan is not in Funded state");
    require(msg.sender != lender, "Lender cannot take the loan");
    require(msg.value == ethCollateralAmount, "Incorrect collateral
amount");
    borrower = payable(msg.sender);
    LoanFactory(factory).recordBorrower(borrower, address(this));
    state = LoanState.Accepted;
    emit LoanTermsAccepted(msg.sender, ethCollateralAmount);
}
```

[i] takeLoan(): Allows the Borrower to take their requested loan amount

```
function takeLoan() external payable {
    require(state == LoanState.Accepted, "Loan is not in accepted state");
    require(msg.sender == borrower, "Only borrower can take the loan");

    state = LoanState.Taken;
    loanTakenTimestamp = block.timestamp; // Store when loan was taken

    LoanFactory factoryContract = LoanFactory(factory);
    factoryContract.updateUtilizationMetrics(loanAmount,true); // Update
utilization metrics in LoanFactory

    borrower.transfer(loanAmount);
    emit LoanTaken(msg.sender, loanAmount);
}
```

[j] repay(): Allows the Borrower to repay the loan amount given they have enough time left

```
function repay() external payable {
    require(state == LoanState.Taken, "Loan is not in Taken state");
    require(msg.sender == borrower, "Only borrower can repay the loan");

    uint256 repaymentAmount = loanAmount + interest + feeAmount;

    require(msg.value == repaymentAmount, "Incorrect repayment amount");
    state = LoanState.Repaid;

    LoanFactory factoryContract = LoanFactory(factory);
    factoryContract.updateUtilizationMetrics(loanAmount,false); // Update
utilization metrics in LoanFactory

    lender.transfer(repaymentAmount);
    borrower.transfer(ethCollateralAmount);

    emit LoanRepaid(msg.sender, repaymentAmount);
}
```

[k] liquidate(): Allows the Lender to liquidate the loan to receive the collateral after time has run out

```
function liquidate() external {
    require(state == LoanState.Taken, "Loan is not in Taken state");
    require(block.timestamp > repayByTimestamp, "Repayment deadline has
not passed");
    require(msg.sender == lender, "Only lender can liquidate the loan");

    state = LoanState.Liquidated;
    lender.transfer(ethCollateralAmount);
    emit LoanLiquidated(borrower, lender, ethCollateralAmount);
}
```

[l] updateRates(): Updates the floating rate for the contract

```
function updateRates() public {
    // Get oracle base rate
    (, int price, , ) = oracle.latestRoundData();
    uint256 baseRate = uint256(price)*100 / 1e6; // Adjust based on the
oracle's decimal places

    // Get utilization rate from LoanFactory
    LoanFactory loanFactory = LoanFactory(factory);
    uint256 utilizationRate = loanFactory.getUtilizationRate();
    uint256 utilizationMultiplier = 15;

    uint256 newFloatingRate = baseRate +
(utilizationMultiplier*utilizationRate); // Adjust based on the oracle's
decimal places
    floatingRate = newFloatingRate;

    uint256 spread = 1;
    fixedRate = floatingRate + spread;
}
```

[m] switchRateType(): Will allow the borrower to switch to the floating rate if they have the fixed rate enabled and vice versa

```
function switchRateType() public {
    require(msg.sender == borrower, "Only borrower can switch rate type");
    if (currentRateType == InterestRateType.Fixed) {
        currentRateType = InterestRateType.Floating;
    } else {
        currentRateType = InterestRateType.Fixed;
    }
}
```

[n] calculateInterest(): Will calculate the interest the user needs to pay

```
function calculateInterest() external {
    require(state == LoanState.Taken, "Loan is not in Taken state");

    if (currentRateType == InterestRateType.Floating) {
        updateRates();
    }

    uint256 interestRate = (currentRateType == InterestRateType.Fixed) ?
fixedRate : floatingRate;
    uint256 timeElapsed = block.timestamp - loanTakenTimestamp;
    if (interestCalculationType == InterestCalculationType.SimpleAPR) {
        calculateSimpleInterest(interestRate, timeElapsed);
    } else if (interestCalculationType ==
InterestCalculationType.CompoundAPY) {
        calculateCompoundInterest(interestRate, timeElapsed);
    }
}
```

[o] calculateSimpleInterest(): If the user selected APR, this formula will be used in calculateInterest()

```
function calculateSimpleInterest(uint256 interestRate, uint256
timeElapsed) private {
    interest = (loanAmount * interestRate * timeElapsed) / (100 * 365
days);
}
```

[p] calculateCompoundInterest(): If the user selected APY, this formula will be used in calculateInterest()

```
function calculateCompoundInterest(uint256 interestRate, uint256
timeElapsed) private {
    uint256 daysElapsed = timeElapsed / 1 days;

    uint256 dailyRate = interestRate / 36500;
    uint256 compoundFactor = 100;

    for (uint i = 0; i < daysElapsed; i++) {
        compoundFactor = (compoundFactor * (100 + dailyRate)) / 100;
    }

    interest = (loanAmount * (compoundFactor - 100)) / 100;
}
```

Appendix B: UI Screenshots

Appendix C: Detailed Test Results

```
PS C:\Users\Bakri\IdeaProjects\Loan Switching Smart Contract\src> npx hardhat test

LoanRequest Contract
  Loan Creation
    ✓ Should initialise loan with correct parameters (1001ms)
  Loan Lifecycle
    ✓ Should allow borrower to accept terms and take loan
    ✓ Should not allow accepting terms with insufficient collateral
  Interest Rate Management
    ✓ Should allow borrower to switch rate types
    ✓ Should not allow non-borrower to switch rates
  Loan Liquidation
    ✓ Should not allow liquidation before deadline
    ✓ Should not allow non-lender to liquidate

LoanFactory Contract
  Proposal Creation
    ✓ Should create a loan proposal correctly (69ms)
    ✓ Should retrieve all open proposals correctly
  Liquidity Management
    ✓ Should add liquidity correctly
    ✓ Should remove liquidity correctly
  Utilization Rate
    ✓ Should calculate utilization rat    ✓ Should calculate utilization rate correctly

12 passing (1s)
```

Appendix D: Deployment Scripts and logs

deploy-factory.js

```
const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.ethers.getSigners();
  console.log("Deploying contracts with the accounts:", deployer.address);

  const loanFactory = await hre.ethers.getContractFactory("LoanFactory");
  const loanFactoryContract = await loanFactory.deploy();

  await loanFactoryContract.waitForDeployment();
  const factoryAddress = await loanFactoryContract.getAddress();

  console.log("LoanFactory deployed to:", factoryAddress);
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

Appendix E. Workplan Summary

Step	Time	Overview
Implement Oracle and Dynamic Switching	25/1/25 – 14/2/25	Will try to implement the Oracle and dynamic switching as well as other listed changes
Frontend Development	14/2/25 – 28/2/25	Develop a user-friendly DApp for users to interact with the smart contract
Testing and Final Adjustments	28/2/25 – 19/3/25	Will run tests of the contracts and DApp. Will also make final adjustments and see what needs to be touched up on

Security Review	20/3/25 – 4/4/25	Will go through the program and try to break it to find security faults to try to fix, so basically, perform a security audit
Final Testing and Deployment	4/4/25 – 21/4/25	Will conduct final testing on contracts and DApp and then deploy on mainnet
Finish Off and Submit	21/4/25 – 30/4/25	Compile all documents and finalise the report

Appendix F. Themes and References

Theme	Description	References
DeFi	Exploration of decentralised financial services which leverage Blockchain	[1]-[6]
Stablecoin in DeFi	Breakdown of stablecoins and the role they are playing for DeFi	[7] – [9]
Lending and Borrowing	Breaking down how lending and borrowing are done in DeFi	[10] – [12]
Interest Rate	Breaking down how interest in DeFi works and how it is calculated	[13] – [18]
Smart Contract	What do smart contracts do, and what is their importance	[19] – [21]
Blockchain	What does Blockchain do, and what different types are there	[22] – [27]

Appendix G. Formulas

(1) APY Formula

$$APY = \left(1 + \left(\frac{r}{n}\right)\right)^n - 1$$

- **r** - nominal interest rate
- **n** - number of compounding periods in a year

(2) APR Formula

$$APR = \left(\frac{r}{n}\right) * n$$

- **r** - nominal interest rate expressed as a decimal

- **n** - number of compounding periods in a year

(3) Utilisation Formula

$$\text{Utilization Rate} = \frac{\text{Total Borrowed}}{\text{Total Liquidity}}$$

Bibliography

- [1] K. Shah, D. Lathiya, N. Lukhi, K. Parmar, and H. Sanghvi, "A systematic review of decentralised finance protocols," *International journal of intelligent networks*, vol. 4, pp. 171–181, Jan. 2023, doi: <https://doi.org/10.1016/j.ijin.2023.07.002>.
- [2] Infosys BPM, "Decentralised Finance (DeFi) and Its Potential to Disrupt Traditional Finance," *Infosysbpm.com*, Jul. 30, 2020. <https://www.infosysbpm.com/blogs/financial-services/decentralised-finance-and-its-potential-to-disrupt-traditional-finance.html> (accessed Jan. 07, 2025).
- [3] R. Sharma, "What Is Decentralised Finance (DeFi) and How Does It Work?," *Investopedia*, Jul. 25, 2024. <https://www.investopedia.com/decentralized-finance-defi-5113835>
- [4] "The Impact of Decentralized Finance on Traditional Banking," *Openware.com*, 2024. <https://www.openware.com/news/articles/the-impact-of-decentralized-finance-on-traditional-banking>
- [5] "What is DeFi? - Decentralised Finance Explained - AWS," *Amazon Web Services, Inc.*, 2020. <https://aws.amazon.com/web3/what-is-defi/>
- [6] "Blocked," *Appinventiv.com*, 2024. <https://appinventiv.com/blog/defi-vs-cefi/>
- [7] A. Hayes, "Stablecoin," *Investopedia*, Jan. 27, 2022. <https://www.investopedia.com/terms/s/stablecoin.asp>
- [8] "What Are Stablecoins and How Do They Work?," *Gemini*. <https://www.gemini.com/cryptopedia/what-are-stablecoins-how-do-they-work>
- [9] Beehive Validator, "The Role of Stablecoins in DeFi: An Overview - Beehive Validator - Medium," *Medium*, May 15, 2023. <https://medium.com/@beehive.validator/the-role-of-stablecoins-in-defi-an-overview-6516461e5084> (accessed Jan. 07, 2025).
- [10] "Stellar | How DeFi Works for Lending and Borrowing Markets," *stellar.org*. <https://stellar.org/learn/lending-and-borrowing-markets>

- [11] "DeFi Lending and Borrowing: a Beginner's Guide," *Coinmetro.com*, 2024. <https://coinmetro.com/learning-lab/defi-lending-and-borrowing> (accessed Jan. 07, 2025).
- [12] "10. DeFi: opportunities, advantages and disadvantages of decentralised Finance," *Kanga University*. <https://kanga.exchange/university/en/courses/beginner-course/lessons/10-defi-opportunities-advantages-and-disadvantages-of-decentralized-finance/>
- [13] "Interest rates in decentralised finance | Banque de France," *Banque de France*, 2024. <https://www.banque-france.fr/en/publications-and-statistics/publications/interest-rates-decentralised-finance>
- [14] Hasib Howlader, "What Is APY In Crypto And How Do You Calculate It?," *Howlader & Co*, Nov. 03, 2022. <https://howladerandco.com/blog/what-is-apy-in-crypto/> (accessed Apr. 26, 2025).
- [15] "APY vs. APR: What's the difference?," *@coinbase*, 2025. <https://www.coinbase.com/en-gb/learn/crypto-basics/apy-vs-apr-what-is-the-difference> (accessed Apr. 26, 2025).
- [16] borzooghaderi, "APY vs APR," *CrowdSwap*, Dec. 18, 2022. <https://crowdswap.org/blog/apr-vs-apy-whats-the-difference/> (accessed Apr. 26, 2025).
- [17] "Aave Interest Rate Model Explained," *Krayondigital.com*, 2024. https://www.krayondigital.com/blog/aave-interest-rate-model-explained?utm_source=chatgpt.com (accessed Apr. 27, 2025).
- [18] I. Macalinao, "Understanding Compound protocol's interest rates," *Ian Macalinao*, Dec. 20, 2020. https://ianm.com/posts/2020-12-20-understanding-compound-protocols-interest-rates?utm_source=chatgpt.com (accessed Apr. 27, 2025).
- [19] "What is a smart contract? " *@coinbase*, 2024. <https://www.coinbase.com/en-gb/learn/crypto-basics/what-is-a-smart-contract>
- [20] E. Canorea, "Smart Contracts: Reasons Why They Are So Important For Business," *Plain Concepts*, Jan. 05, 2023. https://www.plainconcepts.com/smart-contracts/#Why_Should_Companies_Consider_Implementing_Smart_Contracts (accessed Jan. 08, 2025).
- [21] CFI Team, "Smart Contracts," *Corporate Finance Institute*, Dec. 07, 2022. <https://corporatefinanceinstitute.com/resources/valuation/smart-contracts/>
- [22] A. Hayes, "Blockchain Facts: What Is It, How It Works, and How It Can Be Used," *Investopedia*, Sep. 16, 2024. <https://www.investopedia.com/terms/b/blockchain.asp>
- [23] IBM, "What Is Blockchain Technology," *IBM*, 2023. <https://www.ibm.com/topics/blockchain>

[24] A. Ujjawal, "How Does the Blockchain Work?," *GeeksforGeeks*, Aug. 14, 2018. <https://www.geeksforgeeks.org/how-does-the-blockchain-work/>

[25] GeeksForGeeks, "Types of Blockchain," *GeeksforGeeks*, Apr. 27, 2022. <https://www.geeksforgeeks.org/types-of-blockchain/>

[26] A. Shalamai, "Pros, Cons, and Uses Cases of Blockchain Technology | Kindgeek," *KindGeek*, Feb. 2024. <https://kindgeek.com/blog/post/what-you-need-to-know-about-blockchain-pros-and-cons>

[27] "What is Ethereum?," *Blockchain.com*, 2021. <https://www.blockchain.com/learning-portal/tokens/ethereum-explained>

[28] "Aave – Open Source DeFi Protocol," *aave.com*. <https://aave.com/>

[29] "MakerDAO | An Unbiased Global Financial System," *makerdao.com*. <https://makerdao.com/en/>